

BAM CLI Function Guide

Directions for Use

Copyright

© Ericsson AB 2026. All rights reserved. No part of this document may be reproduced in any form without the written permission of the copyright owner.

Disclaimer

The contents of this document are subject to revision without notice due to continued progress in methodology, design and manufacturing. Ericsson shall have no liability for any error or damage of any kind resulting from the use of this document.



Contents

1	BAM CLI Function Guide Introduction	1
2	Prerequisites for BAM CLI	2
3	Download and Setup BAM CLI	3
4	BAM CLI Commands	5
4.1	Flags	7
4.2	Access Management	7
4.3	Alarms	14
4.4	ASIH	20
4.5	Backup-Restore	22
4.6	Backup-Restore-V4	36
4.7	BAM-CM-Agent	57
4.8	Cert-Management-v3	63
4.9	CM-Mediator	76
4.10	Config	84
4.11	Data-Collector	86
4.12	Function-Control	87
4.13	Help	104
4.14	Help-Config	105
4.15	Help-Input	106
4.16	Key Management	107
4.17	Login	116
4.18	Logout	117
4.19	Logs	118
4.20	Metrics	136
4.21	Topology Manager	137
4.22	Extended Data Collection	140
4.23	Trace Management	150





1 BAM CLI Function Guide Introduction

BAM CLI provides an option to download BAM CLI software binary for different OS platforms. BAM CLI service runs in a pod deployed separately. This runs as a web server which is exposed outside the Kubernetes cluster through ingress HTTP proxy. Web server provides an option to download and setup BAM CLI software binary. BAM CLI software binary provides different commands to access data for different services deployed in the Kubernetes cluster.

BAM CLI user, before running any commands, needs to perform a login based on which the user is authenticated with IAM (KeyCloak). Once the user is successfully authenticated, the IAM server responds with the JWT token, which is needed for executing the subsequent commands. For every command executed, the CLI automatically attaches the JWT as the bearer token in the request (each command is executed internally as an HTTP request). Every HTTP request is passed through Ingress and routed to the corresponding service. The Domain Proxy validates this bearer token and authorizes the user. During this authorization process, an audit log entry is created to track the user's access. On successful authorization, the request would be passed on to the service for further processing.

Note: All HTTP requests sent from the bamctl commands (except for login and logout) are sent towards the BAM CLI FQDN.



2 Prerequisites for BAM CLI

The prerequisites in this section need to be in place to be able to use BAM CLI.

- The host from which `bamctl` is executed must have network connectivity to the BAM CLI FQDN.
- The host from which `bamctl` is executed must have network connectivity to the IAM FQDN for user authentication.



3 Download and Setup BAM CLI

To download and setup BAM CLI follow the steps as follows.

Steps

- 1 Download BAM CLI using one of the following options:

- Link available in web page.

```
https://eric-bss-bam-cli.<oamSiteDomainName>
```

Example:

```
https://eric-bss-bam-cli.bam-cluster01.operator.com
```

- Download Linux binary directly using curl:

```
curl -kO https://eric-bss-bam-cli.<oamSiteDomainName>/images/linux/bam-cli
```

Example:

```
curl -kO https://eric-bss-bam-cli.bam-cluster01.operator.com/images/linux/bam-cli
```

- 2 If the Cluster FQDN is not previously configured, add Cluster FQDN information:

```
./bamctl config set-config --key fqdn --value <oamSiteDomainName>
```

Example:

```
./bamctl config set-config --key fqdn --value bam-cluster01.operator.com
```

- 3 If the BAM CLI FQDN macro has been changed from its default value (eric-bss-bam-cli.<oamSiteDomainName>), configure bamctl with the new BAM CLI FQDN value:

```
./bamctl config set-config --key cli-server --value <bamCliFqdn>
```

Example:

```
./bamctl config set-config --key cli-server --value bam-cli.bam-cluster01.operator.com
```



- 4 If the IAM FQDN macro differs from the default value (`eric-sec-access-mgmt.<oamSiteDomainName>`), use the `-t` option with the `bamctl login` command only on the initial login:

```
bamctl login -t https://<bamAccessMgmtFqdn>/auth/realms/master/protocol/ope
```

Example:

```
bamctl login -t https://iam.bam-cluster01.operator.com/auth/realms/master/p
```

Where:

`<oam-site-domain-name>` : value of "Enter your operation and maintenance site domain name" from the **Questionnaire to Prepare Deployment Configuration** in *BAM Installation Guide*, or the value of `oamSiteDomainName` which can be found in `<base-directory>/config/env/prod/environ.yaml`.

`<bamCliFqdn>` : custom value of the `bamCliFqdn` macro which can be found in `<base-directory>/config/env/prod/environ.yaml`. For more information refer to the *BAM Installation Guide* for section *Prepare Deployment Configuration Macros*.

Note: To connect another site (in a multi-site environment), always run the preceding command with another site cluster FQDN information.



4 BAM CLI Commands

The commands supported by BAM CLI are listed in the following table.

Table 1 List of Commands

Command	Description	Reference
access-management	APIs for handling access management	Access Management
alarms	Alarm Handler API	Alarms
asih	Application Sys Info Handler API	ASIH
backup-restore	Backup and Restore	Backup-Restore
backup-restore-v4	Backup and Restore API V4	Backup-Restore-V4
bam-cm-agent	BAM CM Agent API	BAM-CM-Agent
cert-management-v3	Certificate Management API v3	Cert-Management-v3
cm-mediator	Configuration Management Mediator Service	CM-Mediator
config	CLI configuration	Config
data-collector	Diagnostic Data Collector	Data-Collector
function-control	System Function Control	Function-Control
help	Help about any command	Help
help-config	Show CLI configuration help	Help-Config
help-input	Show CLI input help	Help-Input
key-management	Management of encryption keys	Key Management
login	Login for authentication	Login
logout	logout	Logout
logs	Search logs	Logs
metrics	Get Metrics information	Metrics
topology-manager	Topology Manager	Topology Manager
xdc	Extended Data Collection	Extended Data Collection
trace-management	APIs for handling Trace Jobs	Trace Management



```
bamctl help
```

Usage:

```
bamctl [command]
```

Available Commands:

access-management	APIs for handling access management
alarms	Alarm Handler API
asih	Application Sys Info Handler API
backup-restore	Backup and Restore
backup-restore-v4	Backup and Restore API V4
bam-cm-agent	BAM CM Agent API
cert-management-v3	Certificate Management API v3
cm-mediator	Configuration Management Mediator Service
config	CLI configuration
data-collector	Diagnostic Data Collector
function-control	System Function Control Admin API
help	Help about any command
help-config	Show CLI configuration help
help-input	Show CLI input help
key-management	Management of encryption keys
login	Login for authentication
logout	logout
logs	Search logs
metrics	Get Metrics information
topology-manager	BAM Topology Manager API
xdc	Extended Data collection API

Flags:

--dry-run	Dry-Run command
-h, --help	help for bamctl
-o, --output-format string	Output format [json, yaml] (default "json")
-q, --query string	Filter / project results using JMESPath
--raw	Output result of query as raw rather than an escaped string
--server string	Override server URL
--show-headers	Output predefined response headers
--test-env	Run in developer test env
--trace	Enable OpenTelemetry tracing for commands
--verbose	Enable verbose log output
-v, --version	version for bamctl

Use "bamctl [command] --help" for more information about a command.

Note:

- 1 After successfully logging in, the user can optionally use the `--server` parameter with CLI commands to connect to a different server URL.



- 2 The `--server` values would be used temporarily for that session and would not be saved to the `bamctl` configuration file.
- 3 Create User and assign group in IAM to access different services. For more information, refer to the [BAM Installation Guide](#).

4.1 Flags

The following global flags can be used with any `bamctl` command or subcommand.

Distributed Tracing

The `--trace` flag is a global flag that enables OpenTelemetry distributed tracing for any `bamctl` command. When used, it creates and propagates tracing context from the CLI to the server, allowing you to track requests throughout the system. Using this flag with `--verbose` would locally output the trace spans produced by the CLI, and any related tracing error.

Note:

This requires a distributed trace collector to be installed in the BAM deployment, and that the user executing the command is authorized to export traces to the collector.

Show Response Headers

The `--show-headers` flag is a global flag that enables output of predefined response headers.

Note:

`cm-mediator`, `cert-management-v3`, and `xdc` commands support predefined response headers. All other commands currently do not support predefined response headers.

4.2 Access Management

access-management provides options to manage user account in the system.

Usage:

```
bamctl access-management [command]
```

Table 2 Available Commands:

Command	Description
reset-password	Reset a user password



Table 2 Available Commands:

Command	Description
get-legal-warning	Get the current legal warning
update-legal-warning	Update the legal warning
get-privacy-notice	Get the current privacy notice
update-privacy-notice	Update the privacy notice
export-realm-configuration	Export Keycloak realm configuration
import-realm-configuration	Import Keycloak realm configuration

Reset Password

This command resets the user password by providing the old password and current password.

After successfully reset the password, perform a logout from bamctl or GUI.

See the [BAM Network Security Capability Guide](#) for section Password Policies to change a new password.

Reset the password by providing newPassword and oldPassword in the console. This is not the recommended way to change a password because of security.

```
bamctl access-management reset-password <realm> newPassword:<newPassword>, oldPassword:<oldPassword>
```

```
bamctl access-management reset-password master newPassword:xxxxPassword1234$, oldPassword:xxxxPassword5678$
```

Or Create a temporary JSON file and use this file in the reset-password command. Remove the temporary JSON file after resetting the password.

```
/tmp/resetpassword.json
```

```
{
  "oldPassword": "xxxxPassword5678$",
  "newPassword": "xxxxPassword1234$"
}
```

```
bamctl access-management reset-password <realm> < /tmp/resetpassword.json
```

Note:



- 1 After successfully resetting the password, logout from the CLI or GUI and login again to execute any further command.
- 2 Do not execute the reset command with verbose mode.
- 3 If the user is an IAM admin user (kcaadminid), then password must be reset in IAM and in secret.

Get Legal Warning

This command retrieves the current legal warning displayed before login.

```
bamctl access-management get-legal-warning
```

Response:

```
IF YOU ARE NOT AN AUTHORIZED USER, PLEASE EXIT IMMEDIATELY
```

Update Legal Warning

This command updates the legal warning displayed before login.

```
bamctl access-management update-privacy-notice < input.txt
```

Example:

Create a text file with the legal warning content:

```
legal-warning.txt
```

```
IF YOU ARE NOT AN AUTHORIZED USER, PLEASE EXIT IMMEDIATELY.
```

```
bamctl access-management update-legal-warning < legal-warning.txt
```

To **disable** the **legal warning** (no message displayed), execute the command without input as follows. This stores an empty value in the ConfigMap, effectively disabling the notice display.

```
bamctl access-management update-legal-warning
```

Response:



```
{  
  "legalWarning": "Legal warning updated successfully"  
}
```

Get Privacy Notice

This command retrieves the current privacy notice displayed after login.

```
bamctl access-management get-privacy-notice
```

Response:

```
"This system is for authorized users only. All activities may be monitored and
```

Update Privacy Notice

This command updates the privacy notice displayed after login.

```
bamctl access-management update-privacy-notice < input.txt
```

Example:

Create a text file with the privacy notice content:

```
privacy-notice.txt
```

```
All activities may be monitored and recorded. This system is for authorized users
```

```
bamctl access-management update-privacy-notice < privacy-notice.txt
```

To **disable** the **privacy notice** (no message displayed), execute the command without input as follows. This stores an empty value in the ConfigMap, effectively disabling the notice display.

```
bamctl access-management update-privacy-notice
```

Response:



```
{
  "privacyNotice": "Privacy notice updated successfully"
}
```

Notes:

- 1 Legal warning displayed before user login to inform users about system access policies.
- 2 Privacy notice displayed after successful login to inform users about data monitoring and privacy policies.
- 3 Both CLI and GUI interfaces use the same centralized ConfigMap for legal warning display.
- 4 Both legal warnings and privacy notices can be **empty** strings if no content is required - this disables their display.
- 5 Any updates to notices takes a minute to propagate and reflect across CLI/GUI.

Export Realm Configuration

This command exports Keycloak realm configuration including users, roles, clients, groups, ldap, identity providers and authentication flows.

```
bamctl access-management export-realm-configuration <realm-name>
```

```
bamctl access-management export-realm-configuration test-realm
```

Response:

```
{
  "Downloaded filename": "test-realm-export-20260107-220116.json"
}
```

Note:

- 1 The exported file masks all the secrets and credentials.
- 2 Large realms may take time to export depending on the number of configurations.
- 3 Export does not take a backup for LDAP or identity provider users.

Import Realm Configuration

This command imports Keycloak realm configuration from an exported JSON file. Optionally supports security configuration through ConfigMap.



- ConfigMap-based Security: Provide a ConfigMap name in the import request to apply additional security controls during the import process.
- User Password Configuration: Provide specific passwords for users or set a general temporary password for all imported users.
- Validation Rules: Apply custom validation for sensitive components like identity providers and LDAP configurations before import.

```
bamctl access-management import-realm-configuration <realm-name> < <export-file>
or
bamctl access-management import-realm-configuration <realm-name> --config-map-name <config-map-name>
```

```
bamctl access-management import-realm-configuration test-realm < test-realm-export-file
```

Response:

```
{
  "message": "Realm configuration imported successfully"
}
```

```
bamctl access-management import-realm-configuration test-realm --config-map-name <config-map-name>
```

```
{
  "message": "Realm configuration imported successfully"
}
```

Example of Configmap:



```

apiVersion: v1
kind: ConfigMap
metadata:
  name: keycloak-security-config
  namespace: <NAMESPACE>
data:
  # Clients configuration
  clients.json: |
    {
      "clients": [
        {
          "clientId": "authn-proxy",
          "secret_name": "eric-bss-bam-authn-proxy-client-secret",
          "secret_key": "client-secret"
        },
        {
          "clientId": "topology-client",
          "secret_name": "topology-client-secret",
          "secret_key": "password"
        }
      ]
    }

  # Users configuration
  users.json: |
    {
      "users": [
        {
          "user_name": "admin",
          "temporary_password": "TempPass123!"
        },
        {
          "user_name": "testuser",
          "temporary_password": "TestTemp456!"
        },
        {
          "user_name": "developer",
          "temporary_password": "DevTemp789!"
        }
      ]
    }

  # General temporary password
  general-temp-password.json: |
    {
      "general_temp_password": {
        "password": "DefaultTemp2025!"
      }
    }

  # LDAP providers configuration
  ldap-providers.json: |
    {
      "ldap-providers": {
        "bind_credentials": "DefaultLdap2025!"
      }
    }

  # Identity providers configuration

```

**Note:**

- 1 The `--config-map-name` parameter allows specifying security configurations for client secrets, user passwords, and validation rules.
- 2 Create the Configmap before import and set all the values. After import realm configuration, Configmap must be deleted.
- 3 Import operations process components independently - partial failures in one component don't affect others.
- 4 Existing resources may be updated or conflicts resolved based on the import configuration.
- 5 If the user already exists in the system, it is not updated.

4.3 Alarms

Alarms command provides options to view raised alarms in the cluster.

Table 3 List of Commands

Command	Description
list-all-alarms	List all active alarms.
get-alarm	get single alarm details
clear-alarm	Clear single alarm

List All Alarms

Usage:

```
bamctl alarms list-all-alarms [flags]
```

```
bamctl alarms list-all-alarms
```

Response:



```
[
  {
    "additionalInformation": {
      "DC ED error": "DC ED not in use"
    },
    "alarmName": "InternalCertificateCreationFailure",
    "description": "Internal Certificates Cannot be Issued",
    "eventTime": "2020-07-10T04:15:00.376991 UTC",
    "faultyResource": "Internal Certificate Authority",
    "serviceName": "adp_gs_internal_ca",
    "severity": "Critical"
  },
  {
    "alarmName": "CertificateExpiryTimeAlarm",
    "description": "Certificate GlobalSign will expire on 2021-12-15 08:",
    "eventTime": "2020-07-09T13:04:57.408000 UTC",
    "faultyResource": "hodor:GlobalSign",
    "serviceName": "eric-bss-bam-secret-monitor",
    "severity": "Major"
  }
]
```

Outputformat Flag:

The **outputformat** flag must be specified to view the details of the alarms in other format.

Possible values for the **outputformat** flag are:

- FullAlarmList
- ShortAlarmList
- SeveritySummary

FullAlarmList:

The **FullAlarmList** fetches all the details of alarms. It displays history attribute and the alarm type attributes along with the current state of the alarm.

```
bamctl alarms list-all-alarms --outputformat=FullAlarmList
```

Response:



```
bamctl alarms get-alarm CertificateExpiryTimeAlarm
{
  "alarmName": "ExemplAlarm",
  "category": "ProcessingErrorAlarm",
  "code": 14024755,
  "description": "Example Alarm Raised",
  "eventTime": "2022-04-28T01:17:28.484000 UTC",
  "expires": 1278,
  "faultyResource": "eric-example-service.ns-1-system",
  "history": [
    {
      "description": "example config not present",
      "eventTime": "2022-04-27T13:07:45.517000 UTC",
      "severity": "Minor"
    }
  ],
  "probableCause": 158,
  "serviceName": "eric-example-service",
  "severity": "Major",
  "specificProblem": "ExampleAlarm",
  "vendor": 193
},
{
  "alarmName": "NewExemplAlarm",
  "category": "ProcessingErrorAlarm",
  "code": 14024756,
  "description": "License for BAM is not present",
  "eventTime": "2022-04-28T01:07:20.084000 UTC",
  "expires": 1270,
  "faultyResource": "eric-example-service-1.ns-1-system",
  "history": [
    {
      "description": "example not present",
      "eventTime": "2022-04-27T13:07:19.740000 UTC",
      "severity": "Minor"
    }
  ],
  "probableCause": 158,
  "serviceName": "eric-example-service-1",
  "severity": "Major",
  "specificProblem": "ExampleAlarmNew",
  "vendor": 193
}
]
```

For all the alarm commands the following optional parameter can also be set:

- **timeZone**: Display the event time in the specified timezone. The default is the local timezone configured for the Alarm Handler service.



- `timeOutputFormat`: Display the event time in a different format. Supported values are [Normal, Iso8601, Source]. The default value is Iso8601.
- `Normal`: Event time is displayed in YYYY-MM-DDTHH:MM:SS.mmmmmm format followed by a space and a timezone name. For example, 2018-09-10T03:17:05.000679 CET.
- `Iso8601`: Event time is displayed in the YYYY-MM-DDTHH:MM:SS.mmmmmz format, where z is the relative time zone offset in hours and minutes to UTC in the format +hhmm or -hhmm. For example, 2018-09-10T03:17:05.000679+02:00.
- `Source`: Event time is displayed in microseconds since 1 Jan 1970 (minus leap seconds) in UTC. Since the Source is always in UTC, the `timeZone` parameter is not applicable. For example, 1663058715715.

Get Alarm

Usage:

```
bamctl alarms get-alarm alarmName serviceName --faultyresource faultyResource
```

When the alarm has no faulty resource, then only alarm and service names are required.

```
bamctl alarms get-alarm LicenseAlarm eric-bss-bam-function-controller-rbac-p
```

Response:

```
{
  "alarmName": "LicenseAlarm",
  "description": "License for BAM is not present",
  "eventTime": "2024-04-10T04:20:34.035108+00:00",
  "serviceName": "eric-bss-bam-function-controller-rbac-proxy",
  "severity": "Major"
}
```

If alarm has `faultyResource` field then `--faultyresource` must also be added in the command. Use `list-all-alarms` to get the `faultyResource` value.

```
bamctl alarms list-all-alarms
bamctl alarms get-alarm LogConnectionAlarm eric-bss-bam-secret-monitor --fau
```



Note: Use the field `alarmName` and `serviceName` to obtain information about a particular alarm; the [BAM Fault Management Function Guide](#) contains full information about each alarm.

Outputformat Flag:

The **Outputformat** flag must be specified to view the details of the alarms in other format.

Possible values for the **Outputformat** flag are:

- **ShortAlarm:** It displays basic alarms details with only the current state.
- **FullAlarm:** It displays history attribute and the alarm type attributes along with the current state of the alarm.

```
bamctl alarms get-alarm alarmName serviceName --faultyresource faultyResource
```

When the alarm has no faulty resource, then only alarm and service names are required.

```
bamctl alarms get-alarm LicenseAlarm eric-bss-bam-function-controller-rbac-proxy
```

```
{
  "alarmName": "LicenseAlarm",
  "category": "ProcessingErrorAlarm",
  "code": 14024705,
  "description": "License for BAM is not present",
  "eventTime": "2024-04-10T04:20:34.035108+00:00",
  "expires": 1437,
  "probableCause": 158,
  "serviceName": "eric-bss-bam-function-controller-rbac-proxy",
  "severity": "Minor",
  "specificProblem": "License not present",
  "vendor": 193
}
```

If alarm has `faultyResource` field then `--faultyresource` must also be added in the command.

```
bamctl alarms get-alarm LogConnectionAlarm eric-bss-bam-secret-monitor --faulty
```



Note: Use the alarm code to obtain information about a particular alarm; the [BAM Fault Management Function Guide](#) contains full information about each alarm.

Clear Alarm

```
bamctl alarms clear-alarm alarmName serviceName --faultyresource faultyResou
```

When the alarm has no faulty resource, then only alarm and service names are required.

```
bamctl alarms clear-alarm Ref_App_Sample_Alarm eric-bss-ref-tpg
```

Response:

```
{
  "message": "alarm deleted"
}
```

If alarm has faultyResource field then --faultyresource must also be added in the command.

```
bamctl alarms clear-alarm CertificateExpiryTimeAlarm eric-bss-bam-secret-mon
```

Response when the alarm is successfully removed.

```
{ "message": "alarm deleted" }
```

Response when the alarm was not found. May happen if the alarm expired.

```
Error calling operation HTTP 404: {"message":"alarm not found"}
```

```
bamctl alarms clear-alarm CertificateExpiryTimeAlarm eric-bss-bam-secret-mon
```



4.4 ASI H

ASI H (Application Sys Info Handler) provides options to retrieve system information from the Application Sys Info Handler service.

Table 4 List of Commands

Command	Description
get-application-id	Get application instance identity
get-swim	Get SWIM data
get-application-report	Get detailed application report

4.4.1 get-application-id

This command retrieves the application instance identity from the ASI H service.

```
bamctl asih get-application-id
```

Response:

```
{
  "applicationId": "site1"
}
```

4.4.2 get-swim

This command retrieves SWIM (Software Information Management) data from the ASI H service.

```
bamctl asih get-swim
```

Response:



```
{
  "commercialVersion": "3.57",
  "description": "BOS Application Manager",
  "productDate": "1970-01-01T00:00:00Z",
  "productName": "BOS Application Manager",
  "productNumber": "APR9010629",
  "productRevision": "3.57.0",
  "semanticVersion": "3.57.0",
  "type": "Released"
}
```

4.4.3 get-application-report

This command retrieves a detailed application report from the ASIH service.

```
bamctl asih get-application-report
```

Response:

```
{
  "applicationInfo": {
    "productName": "BOS Application Manager",
    "productNumber": "APR9010629",
    "productRevision": "3.57.0",
    "applicationInstanceIdentity": "BOSApplicationManager-APR9010629-3.57.0a"
  },
  "systemInfo": {
    "hostname": "site1",
    "timestamp": "2024-09-29T12:16:28.813+02:00"
  },
  "swimData": {
    "description": "BOS Application Manager provides system information hand",
    "components": [
      {
        "name": "core-component",
        "version": "3.57.0",
        "status": "active"
      }
    ]
  }
}
```



4.5 Backup-Restore

Backup-Restore provides options to perform operations for backup and restore using RBAC proxy of BRO service.

Table 5 List of Commands

Command	Description
execute	Perform Backup and Restore operations.
add-periodic-event	Add periodic event for a backup manager.
delete-periodic-event	Delete a backup-managers periodic event.
list-actions	List all actions for a backup manager.
list-all-backups	List all available backups for the given backup manager.
list-backup-managers	List all available backup managers.
list-periodic-events	List all periodic events for a backup manager.
get-action	Get status of action.
get-backup	Get backup status.
get-housekeeping	Get a backup-managers housekeeping configuration.
get-scheduler-configuration	Get a backup-managers scheduler configuration.
update-periodic-event	Update a backup-managers periodic event.
update-housekeeping	Update a backup-managers housekeeping.
update-scheduler-configuration	Update a backup-managers scheduler configuration.

Execute

Execute command is used to perform different actions like create, delete, restore, import, and export backup using the BRO service.

```
bamctl backup-restore execute --help
```

Response:



```
## Request Schema (application/json)

description: backup action body
properties:
  action:
    enum:
      - CREATE_BACKUP,RESTORE,DELETE_BACKUP,IMPORT,EXPORT
    type: string
  payload:
    $ref: '#/components/schemas/backupname'
required:
- action
- payload

Usage:
  bamctl backup-restore execute backup-manager-id [flags]

Examples:
  bamctl execute backup-manager-id action: IMPORT, payload{password: password}

Flags:
  -h, --help    help for execute

Global Flags:
  --dry-run          Dry-Run command
  -o, --output-format string  Output format [json, yaml] (default "json")
  -q, --query string    Filter / project results using JMESPath
  --raw               Output result of query as raw rather than an array
  --server string      Override server URL
  --test-env          Run in developer test env
  --verbose           Enable verbose log output
```

All the action commands can be performed using two different methods.

- 1 Using direct command
- 2 Using a JSON file

Create Backup:

Method 1:

```
bamctl backup-restore execute <backup-manager-id> action: CREATE_BACKUP,payload
```

Example

```
bamctl backup-restore execute configuration-data action: CREATE_BACKUP,payload
```



Response:

As a response a random unique identifier is returned as `id`.

```
{
  "id": "40659"
}
```

Method 2:

Create a backup using a JSON file. Prepare the JSON file with proper keys and values like illustrated in the following example.

input.json

```
{
  "action": "CREATE_BACKUP",
  "payload": {
    "backupName": "myBackup"
  }
}
```

```
bamctl backup-restore execute configuration-data < input.json
```

As a response a random unique identifier is returned as `id`.

Response:

```
{
  "id": "40659"
}
```

Restore Backup:

Method 1:

```
bamctl backup-restore execute <backup-manager-id> action: RESTORE,payload.backup
```

Example

```
bamctl backup-restore execute configuration-data action: RESTORE,payload.backup
```



Response:

As a response a random unique identifier is returned as id.

```
{
  "id": "7055"
}
```

Method 2:

input.json

```
{
  "action": "RESTORE",
  "payload": {
    "backupName": "myBackup"
  }
}
```

```
bamctl backup-restore execute configuration-data < input.json
```

As a response a random unique identifier is returned as id.

Delete Backup:

Method 1:

```
bamctl backup-restore execute <backup-manager-id> action: DELETE_BACKUP,payload
```

Example

```
bamctl backup-restore execute configuration-data action: DELETE_BACKUP,payload
```

As a response a random unique identifier is returned as id.

```
{
  "id": "29768"
}
```

**Method 2:**

input.json

```
{
  "action": "DELETE_BACKUP",
  "payload": {
    "backupName": "myBackup"
  }
}
```

```
bamctl backup-restore execute configuration-data < input.json
```

Export Backup:**Method 1:**

```
bamctl backup-restore execute <backup-manager-id> action: EXPORT,payload.backupM
```

Example

```
bamctl backup-restore execute configuration-data action: EXPORT,payload.backupM
```

Method 2:

input.json

```
{
  "action": "EXPORT",
  "payload": {
    "backupName": "myBackup",
    "password": "@password",
    "uri": "sftp://xxxx@203.0.113.93:22/tmp/"
  }
}
```

```
bamctl backup-restore execute configuration-data < input.json
```

As a response a random unique identifier is returned as id.



Response:

```
{
  "id": "29769"
}
```

Import Backup:

Method 1:

```
bamctl backup-restore execute <backup-manager-id> action: IMPORT,payload.uri:
```

Example

```
bamctl backup-restore execute configuration-data action: IMPORT,payload.uri:
```

Method 2:

input.json

```
{
  "action": "IMPORT",
  "payload": {
    "backupName": "myBackup",
    "password": "@password",
    "uri": "sftp://xxxx@203.0.113.93:22/tmp/"
  }
}
```

As a response a random unique identifier is returned as id.

Response:

```
{
  "id": "29770"
}
```

Add Periodic Event

This command adds a periodic event for a particular backup-manager, returning the ID of the new event.



```
bamctl backup-restore add-periodic-event <backup-manager-id> startTime: <timestamp>
```

Example

```
bamctl backup-restore add-periodic-event configuration-data startTime: 2022-12-01T00:00:00Z
```

Response:

```
{
  "id": "25563"
}
```

Delete Periodic Event

Delete a backup manager's periodic event.

```
bamctl backup-restore delete-periodic-event <backup-manager-id> <event-id>
```

Example

```
bamctl backup-restore delete-periodic-event configuration-data 25563
```

List Actions

This command provides an option to list all actions for a particular backup manager.

```
bamctl backup-restore list-actions <backup-manager-id>
```

Example

```
bamctl backup-restore list-actions configuration-data
```

Response:



```
{
  "actions": [
    {
      "id": "28694",
      "name": "HOUSEKEEPING",
      "result": "SUCCESS",
      "startTime": "2022-12-20T14:19:03.999111+01:00"
    },
    {
      "id": "27853",
      "name": "CREATE_BACKUP",
      "result": "SUCCESS",
      "startTime": "2022-12-20T14:45:00.323558+01:00"
    }
  ]
}
```

List All Backups

This command gets all available backups for a particular backup-manager id.

```
bamctl backup-restore list-all-backups <backup-manager-id>
```

Example

```
bamctl backup-restore list-all-backups configuration-data
```

Response:



```
{
  "backupManagers": [
    {
      "backupDomain": null,
      "backupType": null,
      "id": "DEFAULT"
    },
    {
      "backupDomain": null,
      "backupType": null,
      "id": "idam-data"
    },
    {
      "backupDomain": null,
      "backupType": null,
      "id": "configuration-data"
    }
  ]
}
```

List Backup Managers

This command gets all the registered backup-managers in the backup-restore service.

```
bamctl backup-restore list-backup-managers
```

Response:



```
{
  "backupManagers": [
    {
      "backupDomain": null,
      "backupType": null,
      "id": "DEFAULT"
    },
    {
      "backupDomain": null,
      "backupType": null,
      "id": "idam-data"
    },
    {
      "backupDomain": null,
      "backupType": null,
      "id": "configuration-data"
    }
  ]
}
```

List Periodic Events

List all periodic events for a particular backup manager.

```
bamctl backup-restore list-periodic-events <backup-manager-id>
```

Example

```
bamctl backup-restore list-periodic-events configuration-data
```

Response:

```
{
  "events": [
    {
      "days": 0,
      "hours": 48,
      "id": "25563",
      "minutes": 0,
      "startTime": "2022-12-16T02:10:00+01:00",
      "weeks": 0
    }
  ]
}
```



Get Action

This command provides an option to check the status of any action using the id value.

```
bamctl backup-restore get-action <backup-manager-id> <id>
```

Example

```
bamctl backup-restore get-action configuration-data 29768
```

Response:

```
{
  "completionTime": "2020-07-13T11:49:08.064",
  "id": "29768",
  "lastUpdateTime": "2020-07-13T11:49:08.064",
  "name": "DELETE_BACKUP",
  "payload": {
    "backupName": "myBackup"
  },
  "progressPercentage": 1,
  "result": "SUCCESS",
  "startTime": "2020-07-13T11:49:07.040",
  "state": "FINISHED"
}
```

Get Backup

This command provides an option to check details for a particular backup using backup-name and backup-manager-id.

```
bamctl backup-restore get-backup <backup-manager-id> <backup-name>
```

Example

```
bamctl backup-restore get-backup configuration-data myBackup
```

Response:



```
{
  "creationTime": "2020-07-13T12:01:51.030",
  "creationType": "MANUAL",
  "id": "myBackup",
  "name": "myBackup",
  "softwareVersions": [
    {
      "agentId": "eric-cm-database-pg",
      "date": "2020-05-18",
      "description": "eric-cm-database-pg Agent for backup and restore.",
      "productName": "Document Database PG",
      "productNumber": "APR 201 34",
      "productRevision": "4.0.0_35",
      "type": "Database"
    },
    {
      "agentId": "APPLICATION_INFO",
      "date": "",
      "description": "",
      "productName": "eric-ctrl-bro",
      "productNumber": "APR 201 40/2",
      "productRevision": "R5A",
      "type": ""
    }
  ],
  "status": "COMPLETE"
}
```

Get Housekeeping

This command retrieves a backup-managers housekeeping configuration.

```
bamctl backup-restore get-housekeeping <backup-manager-id>
```

Example

```
bamctl backup-restore get-housekeeping configuration-data
```

Response:

```
{
  "auto-delete": "enabled",
  "max-stored-manual-backups": 3
}
```



Get Scheduler Configuration

This command retrieves a backup-managers scheduler configuration.

```
bamctl backup-restore get-scheduler-configuration <backup-manager-id>
```

Example

```
bamctl backup-restore get-scheduler-configuration configuration-data
```

Response:

```
{
  "adminState": "UNLOCKED",
  "scheduledBackupName": "SCHEDULED_BACKUP",
  "autoExport": "DISABLED",
  "autoExportPassword": "",
  "autoExportUri": null,
  "mostRecentlyCreatedAutoBackup": null,
  "nextScheduledTime": null
}
```

Response Parameters:

- adminState: Scheduler state - "UNLOCKED" (active) or "LOCKED" (disabled)
- scheduledBackupName: Prefix for scheduled backup names
- autoExport: Auto-export state - "ENABLED" or "DISABLED"
- autoExportPassword: Export authentication password (always hidden)
- autoExportUri: Export destination URI or null if not configured
- mostRecentlyCreatedAutoBackup: Name of latest scheduled backup or null
- nextScheduledTime: ISO timestamp of next scheduled backup or null

Update Periodic Event

This command updates a backup-manager's periodic event, returning the updated event details.

```
bamctl backup-restore update-periodic-event <backup-manager-id> <event-id> start
```

Example

```
bamctl backup-restore update-periodic-event configuration-data 25563 startTime
```



Response:

```
{
  "days": 0,
  "hours": 48,
  "id": "25563",
  "minutes": 0,
  "startTime": "2022-12-16T02:10:00+01:00",
  "weeks": 0
}
```

Update Housekeeping

This command updates a backup-managers housekeeping configuration, returning an action ID.

```
bamctl backup-restore update-housekeeping <backup-manager-id> <max-stored-manu
```

Example

```
bamctl backup-restore update-housekeeping configuration-data max-stored-manu
```

Response:

```
{
  "id": "56829"
}
```

Update Scheduler Configuration

This command updates a backup-managers scheduler configuration.

```
bamctl backup-restore update-scheduler-configuration <backup-manager-id> <co
```

Example

```
bamctl backup-restore update-scheduler-configuration configuration-data admi
```

or



```
bamctl backup-restore update-scheduler-configuration configuration-data < input
```

input.json

```
{
  "adminState": "UNLOCKED",
  "scheduledBackupName": "MY_BACKUP",
  "autoExport": "DISABLED"
}
```

Response:

```
{
  "adminState": "UNLOCKED",
  "scheduledBackupName": "MY_BACKUP",
  "autoExport": "DISABLED",
  "autoExportPassword": "",
  "autoExportUri": null,
  "mostRecentlyCreatedAutoBackup": null,
  "nextScheduledTime": null
}
```

4.6 Backup-Restore-V4

Backup-Restore-V4 provides the command interface for v4 backup and restore operations. The v4 API uses RESTful endpoints with dedicated commands for each operation, replacing the action-based pattern of the v1 API.

Table 6 List of Commands

Command	Description
create-backup	Create a new backup
create-calendar-schedule	Create calendar schedule
create-export	Export a backup
create-import	Import a backup
create-periodic-schedule	Create periodic schedule
create-restore	Start a restore
create-sftp-server	Create SFTP server
delete-backup	Delete a backup
delete-calendar-schedule	Delete calendar schedule



Table 6 List of Commands

Command	Description
delete-periodic-schedule	Delete periodic schedule
delete-sftp-server	Delete SFTP server
get-backup	Get backup details
get-backup-manager	Get specific backup manager
get-calendar-schedule	Get calendar schedule
get-export	Get specific export
get-health	Get orchestrator health
get-housekeeping	Get housekeeping configuration
get-import	Get specific import
get-last-task	Get last task for BRM
get-periodic-schedule	Get periodic schedule
get-restore	Get specific restore
get-scheduling	Get scheduling configuration
get-sftp-server	Get SFTP server
get-task	Get specific task
list-agents	List agents for backup manager
list-backup-managers	List all backup managers
list-backups	List all backups
list-calendar-schedules	List calendar schedules
list-exports	List exports for backup
list-imports	List recent imports
list-periodic-schedules	List periodic schedules
list-restores	List restores for backup
list-sftp-servers	List SFTP servers
list-tasks	List all tasks for BRM
patch-calendar-schedule	Partially update calendar schedule
patch-housekeeping	Partially update housekeeping
patch-periodic-schedule	Partially update periodic schedule
patch-sftp-server	Partially update SFTP server
update-backup-task-state	Update backup task state
update-calendar-schedule	Replace calendar schedule
update-export-task-state	Update export task state



Table 6 List of Commands

Command	Description
update-housekeeping	Replace housekeeping configuration
update-import-task-state	Update import task state
update-periodic-schedule	Replace periodic schedule
update-restore-task-state	Update restore task state
update-scheduling	Replace scheduling configuration
update-sftp-server	Replace SFTP server

4.6.1 get-health

Get the health status of the Backup and Restore Orchestrator.

```
bamctl backup-restore-v4 get-health
```

Response:

```
{
  "availability": "Available",
  "registeredAgents": ["eric-cm-database-pg", "eric-idam-database-pg"],
  "status": "Healthy"
}
```

4.6.2 list-backup-managers

List all registered backup managers.

```
bamctl backup-restore-v4 list-backup-managers
```

Response:

```
{
  "backupManagers": [
    {"id": "DEFAULT"},
    {"id": "configuration-data"},
    {"id": "idam-data"}
  ]
}
```



4.6.3 get-backup-manager

Get details of a specific backup manager including available tasks.

```
bamctl backup-restore-v4 get-backup-manager <backup-manager-id>
```

```
bamctl backup-restore-v4 get-backup-manager configuration-data
```

Response:

```
{
  "availableTasks": [
    {"create": {"href": "/backup-restore/v4/backup-managers/configuration-data/create"},
    {"import": {"href": "/backup-restore/v4/backup-managers/configuration-data/import"}},
  ],
  "id": "configuration-data",
  "ongoingTasks": []
}
```

4.6.4 list-agents

List registered backup agents for a backup manager.

```
bamctl backup-restore-v4 list-agents <backup-manager-id>
```

Response:

```
["eric-cm-database-pg"]
```

4.6.5 create-backup

Create a new backup. Asynchronous operation that returns a task ID.

```
bamctl backup-restore-v4 create-backup <backup-manager-id> < input.json
```

Request Schema (application/json):



```
properties:
  name:
    type: string
    description: Name of the backup to create
required:
- name
```

```
bamctl backup-restore-v4 create-backup configuration-data < input.json
```

input.json

```
{"name": "myBackup"}
```

Response (HTTP 202):

```
{"id": "63003"}
```

4.6.6

list-backups

List all backups for a backup manager.

```
bamctl backup-restore-v4 list-backups <backup-manager-id>
```

Response:

```
[
  {
    "creationTime": "2026-04-14T12:46:31+02:00",
    "creationType": "MANUAL",
    "id": "myBackup",
    "name": "myBackup",
    "status": "COMPLETE"
  }
]
```

4.6.7

get-backup

Get backup details and status.



```
bamctl backup-restore-v4 get-backup <backup-manager-id> <backup-name>
```

```
bamctl backup-restore-v4 get-backup configuration-data myBackup
```

Response:

```
{
  "creationTime": "2026-04-14T12:46:31+02:00",
  "creationType": "MANUAL",
  "id": "myBackup",
  "name": "myBackup",
  "status": "COMPLETE",
  "task": {
    "completionTime": "2026-04-14T12:46:42.458051",
    "progressPercentage": 1,
    "result": "SUCCESS",
    "state": "FINISHED"
  }
}
```

4.6.8 delete-backup

Delete a backup.

```
bamctl backup-restore-v4 delete-backup <backup-manager-id> <backup-name>
```

Response: HTTP 204 No Content.

4.6.9 list-tasks

List all tasks for a backup manager.

```
bamctl backup-restore-v4 list-tasks <backup-manager-id>
```

Response:



```
[  
  {"id": "63003", "name": "CREATE_BACKUP", "result": "SUCCESS", "startTime":  
}
```

4.6.10 **get-task**

Get details of a specific task.

```
bamctl backup-restore-v4 get-task <backup-manager-id> <task-id>
```

Response:

```
{  
  "completionTime": "2026-04-14T12:46:42.458051",  
  "lastUpdateTime": "2026-04-14T12:46:42.458051",  
  "progressPercentage": 1,  
  "resource": "/backup-restore/v4/backup-managers/configuration-data/backups",  
  "result": "SUCCESS",  
  "resultInfo": "{Agent: eric-cm-database-pg, Stage: PREPARATION, success: t",  
  "startTime": "2026-04-14T12:46:29.696468",  
  "state": "FINISHED"  
}
```

4.6.11 **get-last-task**

Get the most recently executed task for a backup manager.

```
bamctl backup-restore-v4 get-last-task <backup-manager-id>
```

Response: Same structure as get-task.

4.6.12 **create-export**

Export a backup to an SFTP server. Asynchronous operation.

```
bamctl backup-restore-v4 create-export <backup-manager-id> <backup-name> <input>
```

Request Schema (application/json):



```
properties:
  uri:
    type: string
    description: SFTP URI for export destination
  password:
    type: string
    description: SFTP password
required:
- uri
- password
```

```
bamctl backup-restore-v4 create-export configuration-data myBackup < input.j
```

```
input.json
```

```
{"uri": "sftp://sftpuser@203.0.113.93:22/exports/", "password": "@password"}
```

Response (HTTP 202):

```
{"id": "17933"}
```

4.6.13

list-exports

List all exports for a backup.

```
bamctl backup-restore-v4 list-exports <backup-manager-id> <backup-name>
```

Response:

```
[
  {"id": "17933", "uri": "sftp://sftpuser@203.0.113.93:22/exports/configur
]
```

4.6.14

get-export

Get export details including task status.



```
bamctl backup-restore-v4 get-export <backup-manager-id> <backup-name> <export-name>
```

Response:

```
{
  "uri": "sftp://sftpuser@203.0.113.93:22/exports/configuration-data/myBackup",
  "task": {
    "additionalInfo": "Exported Backup: myBackup-2026-04-14.tar.gz",
    "completionTime": "2026-04-14T12:47:30.550049",
    "progressPercentage": 1,
    "result": "SUCCESS",
    "state": "FINISHED"
  }
}
```

4.6.15

create-import

Import a backup from an SFTP server. The URI must point to a previously exported .tar.gz file.

```
bamctl backup-restore-v4 create-import <backup-manager-id> < input.json
```

Request Schema (application/json):

```
properties:
  uri:
    type: string
    description: SFTP URI pointing to the exported backup file
  password:
    type: string
    description: SFTP password
required:
- uri
- password
```

```
bamctl backup-restore-v4 create-import configuration-data < input.json
```

input.json



```
{"uri": "sftp://sftpuser@203.0.113.93:22/exports/configuration-data/myBackup"
```

Response (HTTP 202):

```
{"id": "47315"}
```

4.6.16

list-imports

List recent imports for a backup manager.

```
bamctl backup-restore-v4 list-imports <backup-manager-id>
```

Response:

```
[
  {
    "backup": "/backup-restore/v4/backup-managers/configuration-data/bac
    "id": "47315",
    "uri": "sftp://sftpuser@203.0.113.93:22/exports/configuration-data/m
  }
]
```

4.6.17

get-import

Get import details including task status.

```
bamctl backup-restore-v4 get-import <backup-manager-id> <import-id>
```

Response:



```
{
  "backup": "/backup-restore/v4/backup-managers/configuration-data/backups/my",
  "id": "47315",
  "task": {
    "completionTime": "2026-04-14T12:48:10.224316",
    "progressPercentage": 1,
    "result": "SUCCESS",
    "state": "FINISHED"
  }
}
```

4.6.18 create-restore

Start a restore from an existing backup. Asynchronous operation.

```
bamctl backup-restore-v4 create-restore <backup-manager-id> <backup-name>
```

Response (HTTP 202):

```
{"id": "53936"}
```

4.6.19 list-restores

List restores for a backup.

```
bamctl backup-restore-v4 list-restores <backup-manager-id> <backup-name>
```

Response:

```
[{"id": "53936"}]
```

4.6.20 get-restore

Get restore details including task status.

```
bamctl backup-restore-v4 get-restore <backup-manager-id> <backup-name> <restore-id>
```



Response:

```
{
  "task": {
    "completionTime": "2026-04-14T12:48:57.934406",
    "progressPercentage": 1,
    "result": "SUCCESS",
    "resultInfo": "{Agent: eric-cm-database-pg, Stage: PREPARATION, succ
    "state": "FINISHED"
  }
}
```

4.6.21

create-sftp-server

Register an SFTP server configuration. Returns HTTP 201 Created.

```
bamctl backup-restore-v4 create-sftp-server <backup-manager-id> < input.json
```

Request Schema (application/json):



```
properties:
  sftpServerName:
    type: string
  remoteAddress:
    type: string
    description: SFTP server address (sftp://user@host)
  remotePort:
    type: integer
  remotePath:
    type: string
  password:
    type: string
    description: Password auth (mutually exclusive with SSH key fields)
  sshHostKeys:
    type: array
    items:
      type: string
      description: Base64-encoded SSH host keys for server identity verification
  publicKey:
    type: string
    description: Base64-encoded OpenSSH public key
  privateKey:
    type: string
    description: Base64-encoded PEM private key
required:
- sftpServerName
- remoteAddress
- remotePort
- remotePath
```

Note: Provide either `password` for password auth, or `privateKey` + `publicKey` + `sshHostKeys` for SSH key auth. All key values must be base64-encoded. At least one authentication method is required. Providing both password and SSH key fields will result in a 422 error.

```
bamctl backup-restore-v4 create-sftp-server configuration-data < input.json
```

input.json



```
{
  "sftpServerName": "mySFTP",
  "remoteAddress": "sftp://sftpuser@203.0.113.93",
  "remotePort": 22,
  "remotePath": "/exports/bam/",
  "password": "@password"
}
```

4.6.22 **list-sftp-servers**

List all SFTP server configurations. Passwords are masked.

```
bamctl backup-restore-v4 list-sftp-servers <backup-manager-id>
```

Response:

```
[
  {"password": "*****", "remoteAddress": "sftp://sftpuser@203.0.113.93", "
]
```

4.6.23 **get-sftp-server**

Get a specific SFTP server configuration by name.

```
bamctl backup-restore-v4 get-sftp-server <backup-manager-id> <server-name>
```

4.6.24 **patch-sftp-server**

Partially update an SFTP server configuration.

```
bamctl backup-restore-v4 patch-sftp-server <backup-manager-id> <server-name>
```

input.json

```
{"remotePath": "/new-path"}
```



4.6.25 update-sftp-server

Replace an SFTP server configuration (PUT). All fields are required.

```
bamctl backup-restore-v4 update-sftp-server <backup-manager-id> <server-name> <input.json>
```

input.json

```
{
  "sftpServerName": "mySFTP",
  "remoteAddress": "sftp://sftpuser@203.0.113.93",
  "remotePort": 22,
  "remotePath": "/new-exports/",
  "password": "@password"
}
```

Note: Returns HTTP 501 Not Implemented in current BRO version. Use patch-sftp-server for partial updates instead.

4.6.26 delete-sftp-server

Delete an SFTP server configuration.

```
bamctl backup-restore-v4 delete-sftp-server <backup-manager-id> <server-name>
```

Response: HTTP 204 No Content.

4.6.27 get-housekeeping

Get housekeeping configuration.

```
bamctl backup-restore-v4 get-housekeeping <backup-manager-id>
```

Response:

```
{"autoDelete": "enabled", "maxStoredBackups": 1}
```

4.6.28 update-housekeeping

Replace housekeeping configuration (PUT). Both fields required.



```
bamctl backup-restore-v4 update-housekeeping <backup-manager-id> < input.json
```

input.json

```
{"autoDelete": "enabled", "maxStoredBackups": 3}
```

4.6.29

patch-housekeeping

Partially update housekeeping configuration (PATCH).

```
bamctl backup-restore-v4 patch-housekeeping <backup-manager-id> < input.json
```

input.json

```
{"maxStoredBackups": 5}
```

4.6.30

get-scheduling

Get scheduling configuration.

```
bamctl backup-restore-v4 get-scheduling <backup-manager-id>
```

Response:

```
{
  "adminState": "LOCKED",
  "autoExport": "DISABLED",
  "autoExportPassword": "*****",
  "autoExportUri": null,
  "mostRecentlyCreatedAutoBackup": null,
  "nextScheduledTime": null,
  "scheduledBackupNamePrefix": "scheduled-backup"
}
```



4.6.31 update-scheduling

Replace scheduling configuration. Requires `autoExportUri`/`autoExportPassword` or `sftpServerName`.

```
bamctl backup-restore-v4 update-scheduling <backup-manager-id> < input.json
```

`input.json`

```
{
  "adminState": "LOCKED",
  "scheduledBackupNamePrefix": "scheduled-backup",
  "autoExportUri": "sftp://sftpuser@203.0.113.93:22/exports/",
  "autoExportPassword": "@password"
}
```

4.6.32 create-periodic-schedule

Create a periodic backup schedule.

```
bamctl backup-restore-v4 create-periodic-schedule <backup-manager-id> < input.json
```

`input.json`

```
{"startTime": "2026-12-16T02:10:00", "hours": 24}
```

Response (HTTP 201):

```
{"id": "45073"}
```

4.6.33 list-periodic-schedules

```
bamctl backup-restore-v4 list-periodic-schedules <backup-manager-id>
```

Response:



```
{"schedules": [{"days": 0, "hours": 24, "id": "45073", "minutes": 0, "startT
```

4.6.34 **get-periodic-schedule**

```
bamctl backup-restore-v4 get-periodic-schedule <backup-manager-id> <schedule
```

4.6.35 **update-periodic-schedule**

Replace a periodic schedule (PUT).

```
bamctl backup-restore-v4 update-periodic-schedule <backup-manager-id> <sched
```

input.json

```
{"startTime": "2026-12-16T02:10:00", "hours": 48}
```

4.6.36 **patch-periodic-schedule**

Partially update a periodic schedule (PATCH).

```
bamctl backup-restore-v4 patch-periodic-schedule <backup-manager-id> <schedu
```

input.json

```
{"hours": 12}
```

4.6.37 **delete-periodic-schedule**

```
bamctl backup-restore-v4 delete-periodic-schedule <backup-manager-id> <sched
```

Response: HTTP 204 No Content.



4.6.38 create-calendar-schedule

Create a calendar-based backup schedule.

```
bamctl backup-restore-v4 create-calendar-schedule <backup-manager-id> < input.json>
```

Request Schema (application/json):

```
properties:
  scheduleName:
    type: string
  backupName:
    type: string
  scheduleType:
    enum: [calendar]
  startTime:
    type: string
  stopTime:
    type: string
  dayOfWeek:
    enum: [MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY]
  dayOfWeekOccurrence:
    enum: [ALL, FIRST, SECOND, THIRD, FOURTH, LAST]
  time:
    type: string
    description: Time of day (HH:mm:ss)
required:
- scheduleName
- backupName
- scheduleType
- startTime
- stopTime
- dayOfWeek
- dayOfWeekOccurrence
- time
```

input.json



```
{
  "scheduleName": "weekly-monday-backup",
  "backupName": "cal-backup",
  "scheduleType": "calendar",
  "startTime": "2026-12-16T02:10:00",
  "stopTime": "2027-12-16T02:10:00",
  "dayOfWeek": "MONDAY",
  "dayOfWeekOccurrence": "ALL",
  "time": "03:00:00"
}
```

Response (HTTP 201):

```
{"id": "7607"}
```

4.6.39 list-calendar-schedules

```
bamctl backup-restore-v4 list-calendar-schedules <backup-manager-id>
```

4.6.40 get-calendar-schedule

```
bamctl backup-restore-v4 get-calendar-schedule <backup-manager-id> <schedule-id>
```

Response:

```
{
  "dayOfMonth": 0,
  "dayOfWeek": "MONDAY",
  "dayOfWeekOccurrence": "ALL",
  "id": "7607",
  "month": 0,
  "startTime": "2026-12-16T02:10:00+01:00",
  "stopTime": "2027-12-16T02:10:00+01:00",
  "time": "03:00:00"
}
```

4.6.41 update-calendar-schedule

Replace a calendar schedule (PUT).



```
bamctl backup-restore-v4 update-calendar-schedule <backup-manager-id> <schedule>
```

4.6.42 patch-calendar-schedule

Partially update a calendar schedule (PATCH).

```
bamctl backup-restore-v4 patch-calendar-schedule <backup-manager-id> <schedule>
```

input.json

```
{"time": "06:00:00"}
```

4.6.43 delete-calendar-schedule

```
bamctl backup-restore-v4 delete-calendar-schedule <backup-manager-id> <schedule>
```

Response: HTTP 204 No Content.

4.6.44 update-backup-task-state

Cancel a running backup by updating its task state. Only applicable while a backup is in progress.

```
bamctl backup-restore-v4 update-backup-task-state <backup-manager-id> <backup-name>
```

input.json

```
{"state": "CANCEL"}
```

Note: Returns HTTP 404 if no backup task is currently running.

4.6.45 update-export-task-state

Cancel a running export.



```
bamctl backup-restore-v4 update-export-task-state <backup-manager-id> <backu
```

```
input.json
```

```
{"state": "CANCEL"}
```

4.6.46 update-import-task-state

Cancel a running import.

```
bamctl backup-restore-v4 update-import-task-state <backup-manager-id> <impor
```

```
input.json
```

```
{"state": "CANCEL"}
```

4.6.47 update-restore-task-state

Cancel a running restore.

```
bamctl backup-restore-v4 update-restore-task-state <backup-manager-id> <back
```

```
input.json
```

```
{"state": "CANCEL"}
```

4.7 BAM-CM-Agent

BAM CM (Configuration Manager) Agent handles the synchronization of configuration data between Kubernetes clusters.

Table 7 List of Commands

Command	Description
update-cmm-config	Starts synchronizing an existing CM Mediator configuration between Kubernetes clusters.



Table 7 List of Commands

Command	Description
list-configurations	List all application configurations.
get-cmm-config	Returns a list of CM Mediator configurations that are currently synchronized between Kubernetes clusters.
delete-cmm-config	Stops the synchronization of a CM Mediator configuration specified by its configuration name.
get-config	Returns the configuration of the (cluster-)local BAM CM Agent service.
update-config	Replace existing configuration.
export-configuration	Export application configuration
import-configuration	Import application configuration

update-cmm-config

This command provides an option to add a CM Mediator configuration to the sync list used by bam-cm-agent. This command has three fields name, key, and value. Here, the name is a unique value and must correspond to the name of the configuration present in the CM Mediator. Key and value fields are optional.

```
bamctl bam-cm-agent update-cmm-config --help
```

Response:



```
## Request Schema (application/json)
additionalProperties:
  $ref: '#/components/schemas/ReqCMMConfig'
example:
  key: app
  name: foo
  value: bar
  type: object

Usage:
bamctl bam-cm-agent update-cmm-config [flags]

Flags:
-h, --help help for update-cmm-config

Global Flags:
--dry-run Dry-Run command
-o, --output-format string Output format [json, yaml] (default "json")
-q, --query string Filter / project results using JMESPath
--raw Output result of query as raw rather than an escaped JSON string or li
--server string Override server URL
--test-env Run in developer test env
--verbose Enable verbose log output
```

Example to Update Configuration with Name:

```
bamctl bam-cm-agent update-cmm-config name: foo
```

Example to Update Configuration with Name and Key or Value Field:

```
bamctl bam-cm-agent update-cmm-config name: foo, key: app, value: bar
```

List Configuration

This command lists all the services configurations synced by bam-cm-agent.

```
bamctl bam-cm-agent list-configurations
```

Response:



```
{
  "configurations": [
    {
      "name": "eric-bss-ui-settings-manager"
    }
  ]
}
```

Get CMM Config

This command provides an option to get all CM Mediator configurations that is being synced by bam-cm-agent.

```
bamctl bam-cm-agent get-cmm-config --help
```

Example

```
bamctl bam-cm-agent get-cmm-config
```

Delete CMM Config

This command provides an option to delete specific CM Mediator configurations from being synced by bam-cm-agent.

```
bamctl bam-cm-agent delete-cmm-config <configuration-name>
```

Example

```
bamctl bam-cm-agent delete-cmm-config foo
```

Get Config

This command provides an option to get configuration that is being used by bam-cm-agent.

```
bamctl bam-cm-agent get-config --help
```

Response: Get BAM CM Agent configuration Usage:



```
bamctl bam-cm-agent get-config [flags]
```

Flags:

-h, --help help for get-config

Global Flags:

-o, --output-format string Output format [json, yaml] (default "json")

-q, --query string Filter / project results using JMESPath

--raw Output result of query as raw rather than an escaped JSON string or list

--server string Override server URL

--verbose Enable verbose log output

Example

```
bamctl bam-cm-agent get-config
```

Update Config

This command provides an option to update configuration that is being used by bam-cm-agent. All fields in the request body are mandatory.

```
bamctl bam-cm-agent update-config --help
```

Response:



```
## Request Schema (application/json)

additionalProperties:
  $ref: '#/components/schemas/Configuration'
example:
  isPrimary: true
  localStorage: eric-bss-bam-cm-store-rbac-proxy
  logLevel: info
  primaryStorage: eric-bss-bam-cm-store-rbac-proxy
  syncInterval: 5
type: object

Usage:
bamctl bam-cm-agent update-config [flags]

Flags:
-h, --help    help for update-config

Global Flags:
-o, --output-format string    Output format [json, yaml] (default "json")
-q, --query string           Filter / project results using JMESPath
--raw                        Output result of query as raw rather than an escaped string
--server string              Override server URL
--verbose                    Enable verbose log output
```

Example

```
bamctl bam-cm-agent update-config isPrimary: true, localStorage: eric-bss-bam-cm-store-rbac-proxy
```

Export Configuration

This command provides the option to export a service configuration that is synced by bam-cm-agent.

```
bamctl bam-cm-agent export-configuration <application configuration name>
```

Example

```
bamctl bam-cm-agent export-configuration eric-bss-ui-settings-manager
```

Import Configuration

This command provides the option to import a service configuration that is synced by bam-cm-agent.



```
bamctl bam-cm-agent import-configuration <application configuration name> <input>
```

Example

```
bamctl bam-cm-agent import-configuration eric-bss-ui-settings-manager <input>
```

Note: Verify the data before executing the import-configuration.

4.8 Cert-Management-v3

Cert-management-v3 provides options to manage operator-specific certificates and keys used for cluster external communication.

The major concepts are Asymmetric Keys, Trusted Certificates, and CMP Server Groups.

Asymmetric Keys handles the keys and certificates used by microservices for cluster external communication, both clients and servers.

Trusted Certificates handles the trusted certificates (CA certificates) needed to verify the peer in, for example, mTLS.

CMP Server Groups handle the configuration of servers and Certificate Authorities needed for certificate online enrollment using the Certificate Management Protocol (CMP), specified by RFC4210.

Table 8 List of Commands

Command	Description
delete-asymmetric-key	Deletes the asymmetric key with the given name
delete-cmp-server-group	Deletes the CMP server group with the given name
delete-crl	Deletes the certificate revocation list with the given name
delete-trusted-certificates	Delete trusted certificates list with name
download-crls	Triggers download of all online CRLs using the configured URIs
enroll-asymmetric-keys-cmpv2	Starts a CMP initial enrollment and creates an asymmetric-key with certificate as a result
get-asymmetric-key	Gets specific asymmetric key with the given name
get-cmp-server-group	Returns a list of all CMP servers in the given group
get-crl	Returns a certificate revocation list
get-trusted-certificates	List of all trusted certificates with given name



Table 8 List of Commands

Command	Description
install-asymmetric-keys-pkcs12	Decodes the provided PKCS12 container and creates an asymmetric-key
list-all-asymmetric-keys	Get all asymmetric keys
list-cmp-server-groups	Returns a list of all CMP server groups
list-crls	Returns a list of all certificate revocation lists
list-trusted-certificates	List of all trusted certificates
put-cmp-server-group	Creates or updates the specified CMP server group
put-crl	Creates or replaces a certificate revocation list
put-trusted-certificates	Creates or replaces a trusted certificate list with provided trusted certificates
renew-asymmetric-key	Renew asymmetric key with given name using CMP
set-renewal-mode	Set the renewal-mode property of the asymmetric-key

Delete Asymmetric Key

Deletes the asymmetric key with the given name.

Usage:

```
bamctl cert-management-v3 delete-asymmetric-key name [flags]
```

Flags:

<code>--accept string</code>	The content types the client is able to understand
<code>--accept-version string</code>	The requested full semantic version of the resource
<code>-h, --help</code>	help for delete-asymmetric-key

```
bamctl cert-management-v3 delete-asymmetric-key <name>
```

Response:

```
{
  "message": "Deleted asymmetric key <name>"
}
```

Delete CMP Server Group

Deletes the CMP server group with the given name.



Usage:

```
bamctl cert-management-v3 delete-cmp-server-group name [flags]
```

Flags:

<code>--accept string</code>	The content types the client is able to unde
<code>--accept-version string</code>	The requested full semantic version of the r
<code>-h, --help</code>	help for delete-cmp-server-group

```
bamctl cert-management-v3 delete-cmp-server-group <name>
```

Response:

```
{
  "message": "Deleted CMP server group <name>"
}
```

Delete CRL

Deletes the certificate revocation list with the given name.

Usage:

```
bamctl cert-management-v3 delete-crl <name> [flags]
```

Flags:

<code>--accept string</code>	The content types the client is able to unde
<code>--accept-version string</code>	The requested full semantic version of the r
<code>-h, --help</code>	help for delete-crl

Response:

```
{
  "message": "Deleted certificate revocation list <name>"
}
```

Delete Trusted Certificates

Delete trusted certificates list with name.



```
bamctl cert-management-v3 delete-trusted-certificates <name> [flags]
```

Flags:

<code>--accept string</code>	The content types the client is able to understand
<code>--accept-version string</code>	The requested full semantic version of the resource
<code>-h, --help</code>	help for delete-trusted-certificates

Response:

```
{
  "message": "Deleted trusted certificate list <name>"
}
```

Download CRLs

Triggers download of all online CRLs using the configured URIs.

```
bamctl cert-management-v3 download-crls [flags]
```

Flags:

<code>--accept string</code>	The content types the client is able to understand
<code>--accept-version string</code>	The requested full semantic version of the resource
<code>--content-type string</code>	see content value for requestBody
<code>-h, --help</code>	help for download-crls

Response: No response

Enroll Asymmetric Keys CMPv2

Starts a CMP initial enrollment and creates an asymmetric-key with certificate as a result.

```
bamctl cert-management-v3 enroll-asymmetric-keys-cmpv2 < input.json
```

input.json



```
{
  "name": "ldapKey",
  "certificateName": "ldapKey",
  "algorithm": "secp256r1",
  "subject": "C=SE,O=Ericsson,CN=someserialnumber.ericsson.com",
  "subjectAltNames": [
    "DNS:someserialnumber.ericsson.com",
    "IP:145.34.23.123",
    "IP:2001:DB8:8:800:200C:417A",
    "URI:http://my.url.here/",
    "URI:urn:uuid:f81d4fae-7feb-11d0-a765-00bd0c92f7cf7"
  ],
  "password": "mysecreatchallengepassword",
  "cmpServerGroup": "CMP_Server_Group_1",
  "trustedCerts": "cmpTrustedCerts"
}
```

Response:

```
{
  "message": "Enrolled and replaced asymmetric-key 'ldapKey'"
}
```

Get Asymmetric Key

Gets specific asymmetric key with the given name.

Usage:

```
bamctl cert-management-v3 get-asymmetric-key name [flags]
```

Flags:

<code>--accept string</code>	The content types the client is able to unde
<code>--accept-version string</code>	The requested full semantic version of the r
<code>-h, --help</code>	help for get-asymmetric-key

```
bamctl cert-management-v3 get-asymmetric-key 'myServiceKey'
```

Response:



```
{
  "name": "myServiceKey",
  "certificateName": "certificate",
  "certificate": "-----BEGIN CERTIFICATE-----MIIDVjCCAj4CCQCOCsAsUKGbwzANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBOQ",
  "algorithm": "rsa2048",
  "publicKey": "-----BEGIN PUBLIC KEY-----MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBOQ",
  "cmpServerGroup": "Enrollment server group",
  "renewalTime": "2020-08-11T16:00:00Z",
  "trustedCerts": "myServiceTrustedCerts",
  "renewalMode": "automatic"
}
```

Get CMP Server Group

Returns a list of all CMP servers in the given group.

```
bamctl cert-management-v3 get-cmp-server-group <name>
```

Response:

```
[
  {
    "name": "CMP_Server_1",
    "uri": "http://192.168.33.27:8080",
    "caCerts": "CMP_Server_1_CA",
    "certificateAuthority": "CN=someorg.com",
    "priority": 1
  },
  {
    "name": "CMP_Server_2",
    "uri": "http://192.168.33.27:8080",
    "caCerts": "CMP_Server_1_CA",
    "certificateAuthority": "C=SE,O=Ericsson,CN=someserialnumber.ericsson.com",
    "priority": 2
  }
]
```

Get CRL

Returns a certificate revocation list.

```
bamctl cert-management-v3 get-crl 'CRL_name'
```

Response:



```
{
  "name": "CRL_name",
  "refreshInterval": 10,
  "uri": "http://pki.ericsson.se/CertData/EGADRootCA.crl",
  "downloadedCrl": "'-----BEGIN X509 CRL----- MIIDIzCCAQsCAQEwDQYJKoZIhvcNAQ",
  "timestamp": "2024-10-11T16:00:00Z"
}
```

Get Trusted Certificates

List of all trusted certificates with given name.

```
bamctl cert-management-v3 get-trusted-certificates 'ldapTrustCerts'
```

Response:

```
{
  "name": "ldapTrustCerts",
  "description": "Trusted certificates to be used by ldap client",
  "certificates": [
    {
      "name": "myServiceTrustCert1",
      "certificate": "-----BEGIN CERTIFICATE----- MIIDoDCCAoigAwIBAgIJAP4Fgk",
      "crl": [
        "MyCrl1",
        "MyCrl2"
      ],
      "state": "enabled"
    },
    {
      "name": "myServiceTrustCert2",
      "certificate": "-----BEGIN CERTIFICATE----- MIICHTCCAcSgAwIBAgIJAPg8dN",
      "state": "disabled"
    }
  ]
}
```

Install Asymmetric Keys Pkcs12

Decodes the provided PKCS12 container and creates an asymmetric-key.

```
bamctl cert-management-v3 install-asymmetric-keys-pkcs12 < input.json
```

input.json



```
{
  "name": "ldapKey",
  "certificateName": "ldapKey",
  "p12": "MIIEIgIBAzCCA-gGCSqGSIB3DQEHAaCCA9kEggPVMIID0TCCAscGCSqGSIB3DQEHbQCCA",
  "p12Password": "mysecretpassword"
}
```

Response:

```
{
  "message": "Replaced asymmetric-key 'ldapKey'"
}
```

List All Asymmetric Keys

Get all asymmetric keys.

```
bamctl cert-management-v3 list-all-asymmetric-keys
```

Response:

```
[
  {
    "name": "myServiceKey",
    "certificateName": "certificate",
    "certificate": "-----BEGIN CERTIFICATE-----MIIDVjCCAj4CCQCOCsAsUKGbwzANBgk",
    "algorithm": "rsa2048",
    "publicKey": "-----BEGIN PUBLIC KEY-----MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMI",
    "cmpServerGroup": "Enrollment server group",
    "renewalTime": "2020-08-11T16:00:00Z",
    "trustedCerts": "myServiceTrustedCerts",
    "renewalMode": "automatic"
  },
  {
    "name": "ldapKey",
    "certificateName": "certificate",
    "certificate": "-----BEGIN CERTIFICATE-----MIICHTCCAcSgAwIBAgIJAPg8dNhYCv7A",
    "algorithm": "secp256r1",
    "publicKey": "-----BEGIN PUBLIC KEY-----MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgA"
  }
]
```

List CMP Server Groups



Returns a list of all CMP server groups.

```
bamctl cert-management-v3 list-cmp-server-groups
```

Response:

```
[
  {
    "name": "CMP_Server_Group_1",
    "cmpServers": [
      {
        "name": "CMP_Server_1",
        "uri": "http://192.168.33.27:8080",
        "caCerts": "CMP_Server_1_CA",
        "certificateAuthority": "CN=someorg.com",
        "priority": 1
      },
      {
        "name": "CMP_Server_2",
        "uri": "http://2001:DB8:8:800:200C:417A",
        "caCerts": "CMP_Server_1_CA",
        "certificateAuthority": "C=SE,O=Ericsson,CN=someserialnumber.ericsson",
        "priority": 2
      }
    ]
  },
  {
    "name": "CMP_Server_Group_2",
    "cmpServers": [
      {
        "name": "CMP_Server_1",
        "uri": "http://192.168.77.31:8080",
        "caCerts": "CmpServer 1 CA",
        "certificateAuthority": "CN=someorg.com",
        "priority": 1
      },
      {
        "name": "CMP_Server_2",
        "uri": "http://192.168.77.31:8080",
        "caCerts": "CmpServer 1 CA",
        "certificateAuthority": "C=SE,O=Ericsson,CN=someserialnumber.ericsson",
        "priority": 2
      }
    ]
  }
]
```



List CRLs

Returns a list of all certificate revocation lists.

```
bamctl cert-management-v3 list-crls
```

Response:

```
[
  {
    "name": "MyCrl_1",
    "refreshInterval": 10,
    "uri": "http://pki.ericsson.se/CertData/EGADRootCA.crl",
    "downloadedCrl": "'-----BEGIN X509 CRL----- MIIDIZCCAQsCAQEwDQYJKoZIhvcNAQELBQAwTELMAAGA...'",
    "timestamp": "2024-10-11T16:00:00Z"
  },
  {
    "name": "MyCrl_2",
    "crl": "'-----BEGIN X509 CRL----- MIIDIZCCAQsCAQEwDQYJKoZIhvcNAQELBQAwTELMAAGA...'"
  },
  {
    "name": "MyCrl_3",
    "refreshInterval": 24,
    "uri": "ldap://192.168.33.27/ericsson_example_crl.crl"
  }
]
```

List Trusted Certificates

List of all trusted certificates.

```
bamctl cert-management-v3 list-trusted-certificates
```

Response:



```
[
  {
    "name": "myServiceTrustedCerts",
    "description": "Trusted certificates to be used by service",
    "certificates": [
      {
        "name": "myServiceTrustCert1",
        "certificate": "-----BEGIN CERTIFICATE----- MIIDoDCCAoigAwIBAgIJAP4F",
        "crl": [
          "MyCrl1",
          "MyCrl2"
        ],
        "state": "enabled"
      },
      {
        "name": "myServiceTrustCert2",
        "certificate": "-----BEGIN CERTIFICATE----- MIICHTCCAcSgAwIBAgIJAPg8",
        "state": "disabled"
      }
    ]
  },
  {
    "name": "ldapTrustCerts",
    "description": "Trusted certificates to be used by ldap client",
    "certificates": [
      {
        "name": "ldapCA1",
        "certificate": "-----BEGIN CERTIFICATE-----MIICEjCCAb+gAwIBAgIJAMH0x",
        "state": "enabled"
      },
      {
        "name": "ldapCA2",
        "certificate": "-----BEGIN CERTIFICATE-----MIICYTCCAeegAwIBAgIJAPp+e",
        "state": "enabled"
      }
    ]
  }
]
```

Put CMP Server Group

Creates or updates the specified CMP server group.

```
bamctl cert-management-v3 put-cmp-server-group <name> < input.json
```

input.json



```
[
  {
    "name": "CMP_Server_1",
    "uri": "http://192.168.44.88:8080",
    "caCerts": "CMP_Server_1_CA",
    "certificateAuthority": "CN=someorg.com",
    "priority": 1
  },
  {
    "name": "CMP_Server_2",
    "uri": "http://192.168.44.88:8080",
    "caCerts": "CMP_Server_1_CA",
    "certificateAuthority": "C=SE,O=Ericsson,CN=someserialnumber.ericsson.com",
    "priority": 2
  }
]
```

Response:

```
{
  "message": "Replaced CMP server group <name>"
}
```

Put CRL

Creates or replaces a certificate revocation list.

```
bamctl cert-management-v3 put-crl <name> < input.json
```

input.json

```
{
  "uri": "http://pki.ericsson.se/CertData/EGADRootCA.crl",
  "refreshInterval": 24
}
```

Response:

```
{
  "message": "Replaced certificate revocation list <name>"
}
```



Put Trusted Certificates

Creates or replaces a trusted certificate list with provided trusted certificates.

```
bamctl cert-management-v3 put-trusted-certificates <name> < input.json
```

input.json:

```
{
  "description": "Trusted certificates for service",
  "certificates": [
    {
      "name": "myServiceTrustCert1",
      "certificate": "-----BEGIN CERTIFICATE-----\nMIIDoDCCAoigAwIBAgIJAP4Fg
      "crl": [
        "MyCrl1",
        "MyCrl2"
      ]
    },
    {
      "name": "myServiceTrustCert2",
      "certificate": "-----BEGIN CERTIFICATE-----\nMIICHTCCAcSgAwIBAgIJAPg8d
      "state": "disabled"
    }
  ]
}
```

Response:

```
{
  "message": "Replaced trusted certificate list <name>"
}
```

Renew Asymmetric Key

Renew asymmetric key with given name using CMP.

```
bamctl cert-management-v3 renew-asymmetric-key <name> < input.json
```

input.json



```
{
  "algorithm": "secp256r1",
  "subjectAltNames": [
    "DNS:someserialnumber.ericsson.com",
    "IP:145.34.23.123",
    "IP:2001:DB8:8:800:200C:417A",
    "URI:http://my.url.here/",
    "URI:urn:uuid:f81d4fae-7feb-11d0-a765-00bd0c92f7cf7"
  ]
}
```

Response:

```
{
  "message": "Renewed asymmetric-key <name>"
}
```

Set Renewal Mode

Set the renewal-mode property of the asymmetric-key.

```
bamctl cert-management-v3 set-renewal-mode <name> < input.json
```

input.json

```
{
  "renewalMode": "automatic"
}
```

Response:

```
{
  "message": "Set renewalMode to 'automatic'"
}
```

4.9

CM-Mediator

CM Mediator provides options to view and partially modify the stored configurations of CM mediator.



Table 9 List of Commands

Command	Description
create-configuration	Create configuration
get-configuration	Get configuration
delete-configuration	Delete existing configuration
list-all-configurations	List all configurations
patch-configuration	Update part of existing configuration
update-configuration	Update existing configuration

Create Configuration

This command provides the option to create a configuration for an existing schema available in the CM mediator.

```
bamctl cm-mediator create-configuration < input.json
```

input.json



```
{
  "data": {
    "ericsson-data-collector-adp:diagnostic-data-collection": {
      "collector-profile": [
        {
          "collection-window": 60,
          "data-to-collect": [
            "logs"
          ],
          "export-ddb-to": [
            "sftp"
          ],
          "log-plane": [
            "kubelog*",
            "adp-app-logs*",
            "adp-app-asi-logs*"
          ],
          "profile-id": "ApplogsDefaultProfile",
          "profile-type": "collection-profile"
        },
        {
          "collection-window": 60,
          "data-to-collect": [
            "logs",
            "metrics"
          ],
          "export-ddb-to": [
            "sftp"
          ],
          "log-plane": [
            "kubelog*",
            "adp-app-logs*",
            "adp-app-asi-logs*"
          ],
          "profile-id": "ApplogsMetricsDefaultProfile",
          "profile-type": "collection-profile"
        },
        {
          "collection-window": 60,
          "data-to-collect": [
            "metrics"
          ],
          "export-ddb-to": [
            "sftp"
          ],
          "profile-id": "MetricsDefaultProfile",
          "profile-type": "collection-profile"
        },
        {
          "collection-window": 60,
          "data-to-collect": [
            "logs"
          ],
          "export-ddb-to": [
            "sftp"
          ],
          "log-plane": [
            "adp-app-asi-logs*"
          ]
        }
      ]
    }
  }
}
```



Response:

```
{
  "message": "Created configuration 'ericsson-data-collector-adp'"
}
```

Delete Configuration

This command provides the option to delete a configuration from CM mediator. It has one mandatory argument for configuration-name.

```
bamctl cm-mediator delete-configuration ericsson-data-collector-adp
```

Response:

```
{
  "message": "Deleted configuration 'ericsson-data-collector-adp'"
}
```

List All Configurations

This command provides the option to get a list of all configurations available in the CM mediator.

```
bamctl cm-mediator list-all-configurations
```

Response:



```
[
  {
    "name": "ietf-truststore",
    "title": "Store ietf-truststore schema: v1.0"
  },
  {
    "name": "ietf-keystore",
    "title": "Store ietf-keystore schema: v1.0"
  },
  {
    "name": "ericsson-data-collector-adp",
    "title": "diagnostic-data-collector"
  },
  {
    "name": "adp-gs-pm-br",
    "title": "ADP-GS PM Bulk Reporter Configuration"
  }
]
```

Get Configuration

This command provides the option to get a configuration from the CM mediator. It has two optional flags `stdjsonpath` and `jsonpointer`. These flags can be used to filter the configuration.

```
bamctl cm-mediator get-configuration ericsson-data-collector-adp
```

The following example filters the configuration using the `stdjsonpath` flag. This gives the json element present for `action-id=5` in `progress-report` of `ericsson-data-collector-adp` configuration.

```
bamctl cm-mediator get-configuration ericsson-data-collector-adp --stdjsonpath
```

Response:



```
{
  "data": [
    {
      "action-id": 5,
      "result": "success",
      "result-info": "Manual trigger successful",
      "state": "finished",
      "time-action-completed": "2020-08-03T09:20:46+00:00",
      "time-action-started": "2020-08-03T09:20:40+00:00",
      "time-of-last-status-update": "2020-08-03T09:20:46+00:00"
    }
  ],
  "name": "ericsson-data-collector-adp",
  "title": "diagnostic-data-collector"
}
```

The following example filters the configuration using `jsonpointer` flag. It results in the json element corresponding to json pointer path.

```
bamctl cm-mediator get-configuration ericsson-data-collector-adp --jsonpoint
```

Response:



```
{
  "data": [
    {
      "action-id": 1,
      "result": "failure",
      "result-info": "Aborting manual collection, specified config profile could not be found",
      "time-action-completed": "2020-07-28T08:25:34+00:00",
      "time-action-started": "2020-07-28T08:25:34+00:00",
      "time-of-last-status-update": ""
    },
    {
      "action-id": 2,
      "result": "success",
      "result-info": "Manual trigger successful",
      "state": "finished",
      "time-action-completed": "2020-08-03T09:20:46+00:00",
      "time-action-started": "2020-08-03T09:20:40+00:00",
      "time-of-last-status-update": "2020-08-03T09:20:46+00:00"
    },
    {
      "action-id": 3,
      "result": "success",
      "result-info": "Manual trigger successful",
      "state": "finished",
      "time-action-completed": "2020-08-03T09:22:15+00:00",
      "time-action-started": "2020-08-03T09:21:53+00:00",
      "time-of-last-status-update": "2020-08-03T09:22:15+00:00"
    },
    {
      "action-id": 4,
      "result": "success",
      "result-info": "Manual trigger successful",
      "state": "finished",
      "time-action-completed": "2020-08-03T10:24:18+00:00",
      "time-action-started": "2020-08-03T10:24:13+00:00",
      "time-of-last-status-update": "2020-08-03T10:24:18+00:00"
    }
  ],
  "name": "ericsson-data-collector-adp",
  "title": "diagnostic-data-collector"
}
```

Optionally with the `--show-headers` flag, predefined headers received in the responses are printed.

```
bamctl cm-mediator get-configuration ericsson-data-collector-adp --show-headers
```



Response:

```
{
  "headers": {
    "ETag": "6e2753c17566d9a59b86e91ef4cfa123"
  }
}
```

Patch Configuration

This command provides the option for partial modification of a stored configuration in the CM mediator. It has one mandatory argument for configuration-name.

```
bamctl cm-mediator patch-configuration ericsson-data-collector-adp < input.j
```

input.json

```
{
  "baseETag": "6e2753c17566d9a59b86e91ef4cfa123",
  "patch": [
    {
      "op": "replace",
      "path": "/ericsson-data-collector-adp:diagnostic-data-collection/colle
      "value": 30
    }
  ]
}
```

Note:

- 1 The value of the baseETag field is received by running the `bamctl cm-mediator get-configuration` command with the `--show-headers` flag
- 2 The value after collector-profile is the index of the profile which is updated.

Response:

```
{
  "message": "Updated configuration 'ericsson-data-collector-adp'"
}
```

Update Configuration



This command provides the option to update an existing configuration in the CM mediator. It has one mandatory argument for configuration-name.

```
bamctl cm-mediator update-configuration ericsson-data-collector-adp < input.json
```

```
{
  "data": {
    "ericsson-data-collector-adp:diagnostic-data-collection": {
      "data-collector": {
        "administrative-state": "locked",
        "periodic-collector": []
      }
    }
  },
  "title": "diagnostic-data-collector"
}
```

Response:

```
{
  "message": "Updated configuration 'ericsson-data-collector-adp'"
}
```

4.10 Config

Config command provides the functionality to add configurations. Added configuration is persisted in the config.yaml file in the config-directory location (default: \$HOME/.bamctl).

Table 10 List of Commands

Command	Description
list-config	List all configuration
get-config	Get configuration for key
set-config	Set configuration for a command

List Config

Usage:



```
bamctl config list-config [flags]
```

Flags:

```
-h, --help    help for list-config
```

```
bamctl config list-config
```

Response:

```
alarms:
server: https://eric-fh-alarm-handler-rbac-proxy
login:
client_id: AuthorizationClient
token_url: https://eric-sec-access-mgmt/auth/realms/master/protocol/openid-c
```

Get Config

Usage:

```
bamctl config get-config [flags]
```

Flags:

```
-h, --help            help for get-config
-k, --key string      Key for configuratio
```

```
bamctl config get-config --key login.token_url
```

Set Config

Usage:

```
bamctl config set-config [flags]
```

Flags:

```
-h, --help            help for set-config
-k, --key string      Key for configuration
-v, --value string    Value for configuration
-a, --values strings  Values for configuration
```



```
bamctl config set-config --key login.token_url --value 'https://eric-sec-access'
```

4.11 Data-Collector

Data-collector provides options to collect data from the Kubernetes cluster based on input profile.

Table 11 List of Commands

Command	Description
collect	Collect data based on input data collection profile

Usage:

```
bamctl data-collector collect [flags]
```

Flags:

```
-h, --help    help for collect
```

The input data to the command is read from stdin and needs to be in JSON format

```
# Request Schema (application/json)
```

```
description: Input for the collect command
```

```
example:
```

```
  input:
```

```
    ericsson-data-collector-adp:profile: HelmChartValues
```

```
properties:
```

```
  input:
```

```
    type: object
```

Example

```
bamctl data-collector collect < input.json
```

where input.json



```
{
  "input":
    {
      "ericsson-data-collector-adp:profile": "<DDC_profile_name>"
    }
}
```

Note: The DDC_profile_name is created during the installation.

```
bamctl data-collector collect < input.json
```

where input.json

```
{
  "input":
    {
      "ericsson-data-collector-adp:profile": "ApplogsDefaultProfile"
    }
}
```

Response:

```
{
  "ericsson-data-collector-adp:return-value":5
}
```

4.12 Function-Control

Function-control provides options to view and update the function control items.

Table 12 List of Commands

Command	Description
list-all-features	Lists all Features.
get-feature	Gets specific Feature for the provided group and id.
list-features	Lists all Features for the provided group.
set-feature-state	Activates or Deactivates Feature for the provided group and id.
set-feature-flag-state	Activate or Deactivates FeatureFlags for the provided group and id.



Table 12 List of Commands

Command	Description
set-feature-item-validity-state	Activate, Enable, or Disable FeatureItemValidity for provided group, id, and version.
list-license-keys	Get all License keys for the provided product type.
get-feature-flag	Gets specific FeatureFlag for the provided group and id.
get-feature-item-validity	Get FeatureItemValidity for provided group, id, and version.
get-license	Get License for provided group, id, and version.
list-all-exposed-items	Lists all exposed Features and FeatureFlags for which activation or deactivation is allowed.
list-all-feature-flags	Lists all FeatureFlags.
list-all-feature-item-validities	Lists all FeatureItemValidities.
list-all-licenses	Lists all Licenses.
list-feature-flags	List all FeatureFlags for the provided group.
list-feature-item-validities	Lists all FeatureItemValidities for provided group.
list-licenses	Lists all Licenses for provided group.
list-all-conditions	Lists all existing conditions.
get-condition	Get condition by id
update-condition	Update condition by id.
update-condition-targets	Add or Delete targets to condition by id
delete-condition	Delete condition by id
list-all-condition-rules	Lists all existing condition-rules.
get-condition-rule	Get condition rule by id
update-condition-rule	Update condition rule by id
delete-condition-rule	Delete condition rule by id
execute-operation	Execute operation
get-feature-flags-graph	Gets Feature Flags Graph for the Feature provided by group and id.
get-feature-graph	Gets Feature Graph for the provided group and id.

Note:

- Change in the state of the function control resources is only allowed in the primary site.
- The command `get - feature - flag` shows only feature flags that are in the hierarchy structure of the respective TPG.



- The command `list-licenses` shows only licenses that are directly attached to the provided group.

List All Features

Usage:

```
bamctl function-control list-all-features
```

Flags:

```
-h, --help    help for list-all-features
```

Get Feature

Usage:

```
bamctl function-control get-feature group fid [flags]
```

Flags:

```
-h, --help    help for get-feature
```

```
bamctl function-control get-feature ref f0000001
```

List Features

Usage:

```
bamctl function-control list-features group [flags]
```

Flags:

```
-h, --help    help for list-features
```

```
bamctl function-control list-features ref
```

Set Feature State

Usage:



```
bamctl function-control set-feature-state group fid [flags]
```

Flags:

```
-h, --help    help for set-feature-state
```

```
bamctl function-control set-feature-state ref c0000001 < input.json
```

where input.json

```
{  
  "activate": true  
}
```

Set Feature Flag State

Usage:

```
bamctl function-control set-feature-flag-state group fid [flags]
```

Flags:

```
-h, --help    help for set-feature-flag-state
```

```
bamctl function-control set-feature-flag-state ref c0000001 < input.json
```

where input.json

```
{  
  "activate": true  
}
```

Set Feature Item Validity State

Usage:

```
bamctl function-control set-feature-item-validity-state group fid version [flags]
```

Flags:

```
-h, --help    help for set-feature-item-validity-state
```



```
bamctl function-control set-feature-item-validity-state ref fiv000001 1.0.0
```

where input.json

```
{
  "state": "enable"
}
```

list-license-keys

This command provides option to get all License keys for the provided product type.

Usage:

```
Usage:
bamctl function-control list-license-keys productType [flags]
```

```
Flags:
  -h, --help    help for list-features
```

Global Flags:

-o, --output-format string	Output format [json, yaml] (default "json")
-q, --query string	Filter / project results using JMESPath
--raw	Output result of query as raw rather than a
--server string	Override server URL
--verbose	Enable verbose log output

Example

```
bamctl function-control list-license-keys BAM
```

list-all-conditions

This command provides option to list all defined conditions.

Usage:

**Usage:**

```
bamctl function-control list-all-conditions [flags]
```

Flags:

```
-h, --help    help for list-all-conditions
```

Global Flags:

```
--dry-run          Dry-Run command
-o, --output-format string  Output format [json, yaml] (default "json")
-q, --query string      Filter / project results using JMESPath
--raw              Output result of query as raw rather than an escaped string
--server string      Override server URL (default "https://eric-bss-bam.amazonaws.com")
--show-headers      Output predefined response headers
--test-env          Run in developer test env
--trace            Enable OpenTelemetry tracing for commands
--verbose          Enable verbose log output
```

```
bamctl function-control list-all-conditions
```

get-condition

This command provides option to fetch condition by id.

Usage:

```
bamctl function-control get-condition conditionid [flags]
```

Flags:

```
-h, --help    help for get-condition
```

Global Flags:

```
--dry-run          Dry-Run command
-o, --output-format string  Output format [json, yaml] (default "json")
-q, --query string      Filter / project results using JMESPath
--raw              Output result of query as raw rather than an escaped string
--server string      Override server URL (default "https://eric-bss-bam.amazonaws.com")
--show-headers      Output predefined response headers
--test-env          Run in developer test env
--trace            Enable OpenTelemetry tracing for commands
--verbose          Enable verbose log output
```

```
bamctl function-control get-condition c1
```

update-condition



This command provides option to create or update condition by id.



```
Usage:
  bamctl function-control update-condition conditionid [flags]

Flags:
  -h, --help    help for update-condition

Global Flags:
  --dry-run          Dry-Run command
  -o, --output-format string  Output format [json, yaml] (default "json")
  -q, --query string      Filter / project results using JMESPath
  --raw                Output result of query as raw rather than an escaped string
  --server string       Override server URL (default "https://eric-bss-bamctl.com")
  --show-headers        Output predefined response headers
  --test-env            Run in developer test env
  --trace               Enable OpenTelemetry tracing for commands
  --verbose             Enable verbose log output

## Request Schema (application/json)

properties:
  conditionRuleId:
    description: name of the ConditionRule resource
    example: rule01
    type: string
  description:
    description: description of the Condition resource
    example: feature x condition description
    type: string
  id:
    description: id of the Condition
    example: cond01
    type: string
  name:
    description: name of the Condition resource
    example: feature x condition
    type: string
  result:
    default: false
    description: result when condition rule matches [currently always set to true]
    example: true
    nullable: true
    type: boolean
  targetingFlagsList:
    description: list of targeting flags
    example:
      - bam-f00001
    items:
      type: string
    type: array
type: object
```



```
bamctl function-control update-condition c1 .id: c1, .name: bam condition, .
```

delete-condition

This command provides option to delete condition by id.

Usage:

Usage:

```
bamctl function-control delete-condition conditionid [flags]
```

Flags:

```
-h, --help    help for delete-condition
```

Global Flags:

```
--dry-run          Dry-Run command
-o, --output-format string  Output format [json, yaml] (default "json")
-q, --query string    Filter / project results using JMESPath
--raw              Output result of query as raw rather than an es
--server string      Override server URL (default "https://eric-bss-
--show-headers      Output predefined response headers
--test-env          Run in developer test env
--trace            Enable OpenTelemetry tracing for commands
--verbose          Enable verbose log output
```

```
bamctl function-control delete-condition c1
```

update-condition-targets

This command provides option to add or delete targets to condition by id.

Usage:



```

Usage:
  bamctl function-control update-condition-targets conditionid [flags]

Flags:
  -h, --help    help for update-condition-targets

Global Flags:
  --dry-run          Dry-Run command
  -o, --output-format string  Output format [json, yaml] (default "json")
  -q, --query string      Filter / project results using JMESPath
  --raw                Output result of query as raw rather than an escaped string
  --server string       Override server URL (default "https://eric-bss-bamctl.com")
  --show-headers        Output predefined response headers
  --test-env            Run in developer test env
  --trace               Enable OpenTelemetry tracing for commands
  --verbose             Enable verbose log output

## Request Schema (application/json)

properties:
  addedFlags:
    description: list of targeting flags
    example:
      - bam-fiv00001
    items:
      type: string
    type: array
  deletedFlags:
    description: list of targeting flags
    example:
      - bam-fiv00002
    items:
      type: string
    type: array
type: object

```

```

bamctl function-control update-condition-targets c1 .addedFlags:"bam-flag1","bam-flag2"

```

Note: Syntax for list with one element is always `.listname[0]: "flag name"`

list-all-condition-rules

This command provides option to list all defined condition rules.

Usage:



```
Usage:
  bamctl function-control list-all-condition-rules [flags]

Flags:
  -h, --help    help for list-all-condition-rules

Global Flags:
  --dry-run          Dry-Run command
  -o, --output-format string  Output format [json, yaml] (default "json")
  -q, --query string    Filter / project results using JMESPath
  --raw              Output result of query as raw rather than an es
  --server string     Override server URL (default "https://eric-bss-
  --show-headers      Output predefined response headers
  --test-env          Run in developer test env
  --trace             Enable OpenTelemetry tracing for commands
  --verbose           Enable verbose log output
```

```
bamctl function-control list-all-condition-rules
```

get-condition-rule

This command provides option to fetch condition rule by id.

```
Usage:
  bamctl function-control get-condition-rule ruleid [flags]

Flags:
  -h, --help    help for get-condition-rule

Global Flags:
  --dry-run          Dry-Run command
  -o, --output-format string  Output format [json, yaml] (default "json")
  -q, --query string    Filter / project results using JMESPath
  --raw              Output result of query as raw rather than an es
  --server string     Override server URL (default "https://eric-bss-
  --show-headers      Output predefined response headers
  --test-env          Run in developer test env
  --trace             Enable OpenTelemetry tracing for commands
  --verbose           Enable verbose log output
```

```
bamctl function-control get-condition-rule r1
```

update-condition-rule



This command provides option to create or update condition rule by id.



```

Usage:
  bamctl function-control update-condition-rule conditionid [flags]

Flags:
  -h, --help    help for update-condition

Global Flags:
  --dry-run          Dry-Run command
  -o, --output-format string  Output format [json, yaml] (default "json")
  -q, --query string    Filter / project results using JMESPath
  --raw               Output result of query as raw rather than an es
  --server string      Override server URL (default "https://eric-bss-
  --show-headers       Output predefined response headers
  --test-env           Run in developer test env
  --trace              Enable OpenTelemetry tracing for commands
  --verbose            Enable verbose log output

## Request Schema (application/json)

properties:
  description:
    description: description of the TargetingRule resource
    example: feature x targeting rule description
    nullable: true
    type: string
  id:
    description: id of the TargetingRule
    example: rule01
    type: string
  key:
    description: context key for evaluation
    example: appGroup
    type: string
  name:
    description: name for the targeting rule
    example: feature x targeting rule
    type: string
  operation:
    description: comparison operation to perform against the key values
    example: in, notIn
    type: string
  result:
    default: false
    description: result when match (always true)
    example: true
    type: boolean
  values:
    description: description of the TargetingRule resource
    example:
      - cha1
      - cha2
    items:
      type: string
    type: array
type: object

```



```
bamctl function-control update-condition-rule r1 .id: r1, .key: location, .open
```

Note: Syntax for list with one element is always `.listname[0]: "values"`.

delete-condition-rule

This command provides option to delete condition rule by id.

Usage:

```
bamctl function-control delete-condition-rule ruleid [flags]
```

Flags:

```
-h, --help    help for delete-condition-rule
```

Global Flags:

<code>--dry-run</code>	Dry-Run command
<code>-o, --output-format string</code>	Output format [json, yaml] (default "json")
<code>-q, --query string</code>	Filter / project results using JMESPath
<code>--raw</code>	Output result of query as raw rather than an escaped string
<code>--server string</code>	Override server URL (default "https://eric-bss-bam")
<code>--show-headers</code>	Output predefined response headers
<code>--test-env</code>	Run in developer test env
<code>--trace</code>	Enable OpenTelemetry tracing for commands
<code>--verbose</code>	Enable verbose log output

```
bamctl function-control delete-condition-rule r1
```

get-feature-graph

This command provides option to fetch feature graph with all levels.

Usage:

**Usage:**

```
bamctl function-control get-feature-graph group fid [flags]
```

Flags:

```
--filterbystatus    filter by status of the Feature flag
-h, --help           help for get-feature-flags-graph
--showstatus         show status of the Feature flag
```

Global Flags:

```
--dry-run           Dry-Run command
-o, --output-format string  Output format [json, yaml] (default "json")
-q, --query string      Filter / project results using JMESPath
--raw                Output result of query as raw rather than an es
--server string        Override server URL (default "https://eric-bss-
--show-headers        Output predefined response headers
--test-env            Run in developer test env
--trace              Enable OpenTelemetry tracing for commands
--verbose            Enable verbose log output
```

Examples:

```
bamctl function-control get-feature-graph sys feat
```

```
{
  "flags": [
    "sys-feat1",
    "sys-feat2"
  ],
  "resourceName": "sys-feat-2.0.0"
}
```

```
bamctl function-control get-feature-graph sys feat --showstatus
```

```
{
  "flags": [
    "sys-feat1:true",
    "sys-feat2:false"
  ],
  "resourceName": "sys-feat-2.0.0"
}
```



```
bamctl function-control get-feature-graph sys feat --filterbystatus=true
```

```
{
  "flags": [
    "sys-feat1"
  ],
  "resourceName": "sys-feat-2.0.0"
}
```

```
bamctl function-control get-feature-graph sys feat --filterbystatus=false
```

```
{
  "flags": [
    "sys-feat2"
  ],
  "resourceName": "sys-feat-2.0.0"
}
```

get-feature-flags-graph

This command provides option to fetch all feature flags from one feature.

Usage:

```
Usage:
  bamctl function-control get-feature-flags-graph group fid [flags]

Flags:
  --filterbystatus      filter by status of the Feature flag
  -h, --help            help for get-feature-flags-graph
  --showstatus          show status of the Feature flag

Global Flags:
  --dry-run              Dry-Run command
  -o, --output-format string Output format [json, yaml] (default "json")
  -q, --query string      Filter / project results using JMESPath
  --raw                 Output result of query as raw rather than an escaped string
  --server string         Override server URL (default "https://eric-bss-bam")
  --show-headers         Output predefined response headers
  --test-env             Run in developer test env
  --trace               Enable OpenTelemetry tracing for commands
  --verbose              Enable verbose log output
```



Examples:

```
bamctl function-control get-feature-flags-graph sys feat1
```

```
{
  "flags": [
    "sys-feat1",
    "sys-feat2"
  ]
}
```

```
bamctl function-control get-feature-flags-graph sys feat --showstatus
```

```
{
  "flags": [
    "sys-feat1:true",
    "sys-feat2:false"
  ]
}
```

```
bamctl function-control get-feature-flags-graph sys feat --filterbystatus=true
```

```
{
  "flags": [
    "sys-feat1"
  ]
}
```

```
bamctl function-control get-feature-flags-graph sys feat --filterbystatus=false
```

```
{
  "flags": [
    "sys-feat2"
  ]
}
```



execute-operation

This command provides option to execute operations on feature items.

Currently, this command is not activated in backend and only syntax is validated.

Usage:

Usage:

```
bamctl function-control execute-operation [flags]
```

Flags:

```
--action string
-h, --help          help for execute-operation
--target string
--type string
```

Global Flags:

```
--dry-run          Dry-Run command
-o, --output-format string  Output format [json, yaml] (default "json")
-q, --query string    Filter / project results using JMESPath
--raw              Output result of query as raw rather than an escaped string
--server string      Override server URL (default "https://eric-bss-bam.amazonaws.com")
--show-headers      Output predefined response headers
--test-env          Run in developer test env
--trace            Enable OpenTelemetry tracing for commands
--verbose          Enable verbose log output
```

```
bamctl function-control execute-operation --action activate --target bam-feature-item
```

4.13 Help

The help command displays a list of commands and helps for a given command.

If called without a parameter, the help command lists all available commands. When called with a command supported by the CLI, a detailed description of the command is displayed.

```
bamctl [command] --help
```

Example

```
bamctl config --help
```



Response:

```
Available Commands:
get-config  get configuration for key
list-config list all configuration
set-config  Set configuration for a command
```

4.14 Help-Config

The `help-config` command displays information about CLI configuration help.

Configuration for the CLI comes from the following places:

- Command options
- Environment variables
- Configuration files

Global Command Option

Command options are passed when invoking the command.

Table 13 Command Options

Name	Type	Description
output-format	string	Output format [json, yaml]
query	string	Filter / project results using JMESPath
raw	bool	Output result of query as raw rather than an escaped JSON string or list
server	string	Override server URL
verbose	bool	Enable verbose log output

Example

`--verbose` configures the CLI to run with additional output for debugging.

Environment Variables

Environment variables must be capitalized, prefixed with `BAMCTL`, and words are separated by an underscore rather than a dash.

Example

Setting `BAMCTL_VERBOSE=1` is equivalent to passing `--verbose` to the command.

Configuration Files



Configuration files can be used to configure the CLI and can be written using JSON, YAML or TOML.

The CLI searches in your home directory first, for example `/home/ebatsum/.bamctl/config.json` and on Mac/Linux also looks in `/etc/bamctl/config.json`

Example

The following is equivalent to passing `--verbose` to the command:

```
{
  "verbose": true
}
```

4.15 Help-Input

Input to the CLI is handled through parameters, arguments, and standard input. The help for an individual command shows the available optional parameters and required arguments.

For requests that require a body, standard input and a CLI shorthand can complement each other to supply the request data.

Standard Input

Standard input allows the user to send in whatever data is required to make a successful request against the API.

Example

```
'my-cli command <input.json' or 'echo '{"hello": "world"}' | my-cli command
```

CLI Shorthand Syntax

Any argument beyond those that are required for a command is treated as CLI shorthand and used to generate structured data for requests. Shorthand objects are specified as key and value pairs. They complement standard input so can be used to override or to add additional fields as needed.

Example

```
'my-cli command <input.json field: value, other: value2'.
```



4.16 Key Management

key-management provides commands to manage Encryption keys in the system.

Usage:

```
bamctl key-management [command]
```

Available Commands:

archive-key	Archive a key version
create-key	Create a new key alias
delete-key	Delete entire key
delete-key-version	Delete key version
get-config	Get Gateway configuration in use
get-key	Retrieve latest encryption key for key alias
get-key-version	Retrieve version specific encryption key for
get-sync-filter-config	Get sync filter configuration
list-all-keys	List all key aliases for a particular Owner
list-key-versions	list all versions of the key alias
revoke-key	Revoke latest key version
rotate-key	Rotate a key to new version
unarchive-key	Unarchive a key version
update-config	Update kms-gateway configuration like default
update-sync-filter-config	Update sync filter configuration

archive-key

This command provides the option to archive a key version.

Usage:

```
bamctl key-management archive-key owner-id key-alias version [flags]
```

Flags:

-h, --help	help for archive-key
--tenant-id string	Specify the tenant ID or tenant name

```
bamctl key-management archive-key eric-bss-bam key1 1000001
```

Response:

```
"Version archived successfully"
```

create-key



This command provides the option to create a new key alias.

Usage:

```
bamctl key-management create-key owner-id key-alias [flags]
```

Flags:

-h, --help	help for create-key
--tenant-id string	Specify the tenant ID or tenant name

```
bamctl key-management create-key eric-bss-bam key1
```

Response:

```
"Key created successfully"
```

delete-key

This command provides the option to delete an entire key.

Usage:

```
bamctl key-management delete-key owner-id key-alias [flags]
```

Flags:

-h, --help	help for delete-key
--tenant-id string	Specify the tenant ID or tenant name

```
bamctl key-management delete-key eric-bss-bam key1
```

Response:

```
"Key deleted successfully"
```

delete-key-version

This command provides the option to delete a key version.

Usage:



```
bamctl key-management delete-key-version owner-id key-alias version [flags]
```

Flags:

```
-h, --help          help for delete-key-version
--tenant-id string  Specify the tenant ID or tenant name
```

```
bamctl key-management delete-key-version eric-bss-bam key1 1000001
```

Response:

```
"Key version deleted successfully"
```

get-config

This command provides the option to get KMS-Gateway configuration in use.

Usage:

```
bamctl key-management get-config
```

```
bamctl key-management get-config
```

Response:

```
{
  "key_rotation_period": 90
}
```

get-key

This command provides the option to get the latest encryption key for key alias.

Usage:



```
bamctl key-management get-key owner-id key-alias [flags]
```

Flags:

-h, --help	help for get-key
--tenant-id string	Specify the tenant ID or tenant name

```
bamctl key-management get-key eric-bss-bam key1
```

Response:

```
{
  "key": "MRKZTA61z+3V1k0zzKpcDZAi0SdYVadfdDiATHiwAWM=",
  "version": 1000001
}
```

get-key-version

This command provides the option to get the version-specific encryption key for the key alias.

Usage:

```
bamctl key-management get-key-version owner-id key-alias version [flags]
```

Flags:

-h, --help	help for get-key-version
--tenant-id string	Specify the tenant ID or tenant name

```
bamctl key-management get-key-version eric-bss-bam key1 1000001
```

Response:

```
{
  "key": "MRKZTA61z+3V1k0zzKpcDZAi0SdYVadfdDiATHiwAWM=",
  "version": 1000001
}
```

get-sync-filter-config

This command provides the option to list the sync filters.



Usage:

```
bamctl key-management get-sync-filter-config [flags]
```

Flags:

-h, --help	help for get-key-version
------------	--------------------------

```
bamctl key-management get-sync-filter-config
```

Response:

```
{
  "exclude": [],
  "include": []
}
```

list-all-keys

This command provides the option to list all key aliases for a specific owner.

Usage:

```
bamctl key-management list-all-keys owner-id [flags]
```

Flags:

-h, --help	help for list-all-keys
--list string	Optional flag to list keys
--tenant-id string	Specify the tenant ID or tenant name

```
bamctl key-management list-all-keys eric-bss-bam
```

Response:

```
{
  "keysInfo": [
    "key1",
    "key2"
  ]
}
```



list-key-versions

This command provides the option to list all versions of a key alias.

Usage:

```
bamctl key-management list-key-versions owner-id key-alias [flags]
```

Flags:

-h, --help	help for list-key-versions
--siteid string	Optional flag to list key versions of a specific site
--tenant-id string	Specify the tenant ID or tenant name

```
bamctl key-management list-key-versions-key eric-bss-bam key1
```

Response:

```
[
  {
    "allow_plaintext_backup": true,
    "auto_rotate_period": 15552000,
    "deletion_allowed": false,
    "derived": false,
    "exportable": true,
    "imported_key": false,
    "keys": {
      "1000001": 1727872889
    },
    "latest_version": 1000001,
    "min_available_version": 1000000,
    "min_decryption_version": 1000001,
    "min_encryption_version": 1000000,
    "name": "key1",
    "type": "aes256-gcm96"
  }
]
```

revoke-key

This command provides the option to revoke a specific key version.

Usage:



```
bamctl key-management revoke-key owner-id key-alias version [flags]
```

Flags:

-h, --help	help for revoke-key
--tenant-id string	Specify the tenant ID or tenant name

```
bamctl key-management revoke-key eric-bss-bam key1 1000001
```

Response:

```
"Key revoked successfully"
```

rotate-key

This command provides the option to rotate a key to a new version.

Usage:

```
bamctl key-management rotate-key owner-id key-alias [flags]
```

Flags:

-h, --help	help for rotate-key
--tenant-id string	Specify the tenant ID or tenant name

```
bamctl key-management rotate-key eric-bss-bam key1
```

Response:

```
"Key rotated successfully"
```

unarchive-key

This command provides the option to unarchive a key version.

Usage:



```
bamctl key-management unarchive-key owner-id key-alias version [flags]
```

Flags:

```
-h, --help          help for unarchive-key
--tenant-id string  Specify the tenant ID or tenant name
```

```
bamctl key-management unarchive-key eric-bss-bam key1 1000001
```

Response:

```
"Version unarchived successfully"
```

update-config

This command provides the option to update the KMS-Gateway configuration in use.

Usage:

```
bamctl key-management update-config [flags]
```

Request Schema (application/json)

```
key_rotation_period:
  description: Defines the custom auto rotate period in days
  example: '90'
  minimum: 1
  maximum: 365
  type: integer
```

Flags:

```
-h, --help  help for update-config
```

```
bamctl key-management update-config < input.json
```

input.json

```
{
  "key_rotation_period": 180
}
```

update-sync-filter-config



This command provides the option to update the KMS-Gateway sync filter configuration in use.

Usage:

```
bamctl key-management update-sync-filter-config [flags]
  Filters:
    type: object
    description: List of filters
    properties:
      include:
        type: array
        description: List of include filters
        items:
          $ref: '#/components/schemas/Filter'
      exclude:
        type: array
        description: List of exclude filters
        items:
          $ref: '#/components/schemas/Filter'
  Filter:
    type: object
    description: A filter object containing a combination of owner, tenant
    properties:
      name:
        type: string
        description: Combination of owner, tenant, and alias

Flags:
  -h, --help    help for update-sync-filter-config
```

```
bamctl key-management update-sync-filter-config < input.json
```

input.json



```
{
  "include": [
    {
      "name": "eric-bss-bam/bam/key1"
    },
    {
      "name": "eric-bss-bam/bam/key2"
    }
  ],
  "exclude": [
    {
      "name": "eric-bss-bam/bam/key3"
    }
  ]
}
```

4.17 Login

Login command authenticates the user with keycloak API and persists the refresh token from the response output which is used to get token during pre-run phase of other commands. Based on token user is authorized to access different services within cluster. It also displays a notice to the user and administrator which is fetched from the BAM CLI server.

When the Multi-Factor Authentication is enabled, and its users first-time login, the setup has to be done from BAM Application Portal. Refer to the [BAM Application Portal Function Guide](#).

After the setup, the user must provide an OTP parameter in addition to the username in the login command.

Usage:

```
bamctl login [flags]
```

Flags:

-c, --client-id string	Client issuing token (default "AuthorizationClient")
-h, --help	help for login
-n, --notice-url string	Notice URL (default "https://<eric-sec-access-mgmt>")
-O, --otp string	Provide OTP if system is enabled with OTP flow
-p, --password-file string	Path for password file
-s, --silent	Skip notice to user and administrators
-t, --token-url string	Token issue URL (default "https://<eric-sec-access-mgmt>")
-u, --username string	Provide username



```
bamctl login --username <username> --password-file <path-to-password-file>
```

Example

```
bamctl login --username <user> --password-file <path-to-password-file> -t ht
```

After successful login refresh token is persisted in the config-directory (\$HOME/.<app-name>) which would be used to get new token for other commands in pre-run phase and this new token is sent as a Bearer token in Authorization header with the request.

Login command also has optional flags for client-id, token-url, notice-url, silent, and otp which can be overridden using --client-id, --token-url, --notice-url, --silent, --otp (-c, -t, -n, -s, -o) respectively.

Table 14 Optional Flags for Login

client-id	Client issuing token (default "AuthorizationClient")
token-url	Token issue URL (default " https://eric-sec-access-mgmt/auth/realms/master/protocol/openid-connect/token ")
notice-url	Notice URL (default " https://eric-bss-bam-cli-proxy/notice ")
silent	Skip the notice to user and administrator
otp	Mandatory to provide OTP if system is enabled with Multi-Factor Authentication

Note: A warning message prints in the console to inform the user if the version is mismatched between bamctl and server.

4.18 Logout

Logout command provides the functionality to logout from the CLI. It would remove the saved user token and CA Certificates.

Example

```
bamctl logout
```

Response:



```
Logout Successful
```

4.19 Logs

Logs provide options to search logs information through RBAC proxy of a search engine service.

Table 15 List of Commands

Command	Description
get-all-indexes	Provide all open indexes.
query	Results all logs information that matches the query entered.
search	Results all logs information matching the request body.

Get All Indexes

Usage:

```
bamctl logs get-all-indexes [flags]
```

Flags:

```
--expand-wildcards string
-h, --help                help for get-all-indexes
```

Example to Fetch a List of Index Names:

```
bamctl logs get-all-indexes -q *.settings.index.provided_name
```

Response:

```
[
  "auditlog-2020.07.15",
  "kubelog-2020.07.15",
  "kubelog-2020.07.16"
]
```

Query to Search



This command provides an option to search logs based on the query parameter. It accepts two mandatory parameters index (logplane) and query (query string to search). It has an optional `--size` parameter to limit the logs output.

Usage:

```
bamctl logs query <index-name> <query-to-search> [flags]
```

Flags:

```
-h, --help          help for query  
-p, --prettyPrint    Reformat output  
--size string
```

Example

```
bamctl logs query kubelog* "xdc-bridge" --size 1
```

Response:



```
{
  "_shards": {
    "failed": 0,
    "skipped": 0,
    "successful": 10,
    "total": 10
  },
  "hits": {
    "hits": [
      {
        "_id": "aZJeg44BqrM909Sp1d8A",
        "_index": "kubelog-2024.03.28",
        "_score": 24.13278,
        "_source": {
          "@timestamp": "2024-03-28T03:25:07.705Z",
          "@version": "1",
          "logplane": "kubelog",
          "message": "[listener.go:57]: Starting TLS listener at: :4141, with c",
          "metadata": {
            "container_name": "xdcbridge",
            "namespace": "namespace",
            "node_name": "node-10-210-166-43",
            "pod_name": "eric-bss-bam-xdc-bridge-75f9955dd9-xnvpd",
            "pod_uid": "691b428d-1362-41a3-887d-0e675f7a252f"
          },
          "service_id": "eric-bss-bam-xdc-bridge",
          "severity": "info",
          "timestamp": "2024-03-28T04:25:07.704+01:00",
          "version": "1.2.0"
        }
      },
      {
        "service_id": "eric-bss-bam-xdc-bridge",
        "severity": "info",
        "timestamp": "2024-03-28T04:25:07.704+01:00",
        "version": "1.2.0"
      }
    ],
    "max_score": 24.13278,
    "total": {
      "relation": "eq",
      "value": 7091
    }
  },
  "timed_out": false,
  "took": 157
}
```

Example to Print the Response in Pretty Format:

Pretty print optional parameters `-p` or `--prettyPrint` is used.

Usage:



```
bamctl logs query kubelog* "MULTILINE" -p
```

Response:



```
{
  "_shards": {
    "failed": 0,
    "skipped": 0,
    "successful": 5,
    "total": 5
  },
  "hits": {
    "hits": [
      {
        "_id": "tR_4VIMB8AMBzEfdN6l8",
        "_index": "kubelog-2022.09.19",
        "_score": 1.3087101,
        "_source": {
          "@timestamp": "2022-09-19T08:58:15.431Z",
          "@version": "1",
          "kubernetes": {
            "replicaset": {
              "name": "eric-bss-ref-tpg-6987d6c995"
            }
          },
          "logplane": "kubelog",
          "message": "Example BAM-MULTILINE exception logging for BAM int
at java.base/jdk.internal.util.Preconditions.outOfBounds(Preconditions.java
at java.base/jdk.internal.util.Preconditions.outOfBoundsCheckIndex(Precondi
at java.base/jdk.internal.util.Preconditions.checkIndex(Preconditions.java
          "metadata": {
            "category": "BAM-tests",
            "container_name": "reftpg",
            "namespace": "namespace",
            "node_name": "node-10-210-166-51",
            "pod_name": "eric-bss-ref-tpg-6987d6c995-7dkkn",
            "pod_uid": "917038a0-626a-4991-934f-2ca177652ed0"
          },
          "service_id": "eric-bss-bam-ref-tpg",
          "severity": "error",
          "timestamp": "2022-09-19T08:58:15.430Z",
          "version": "0.2.0"
        },
        "_type": "_doc"
      }
    ],
    "max_score": 1.3513527,
    "total": {
      "relation": "eq",
      "value": 2
    }
  },
  "timed_out": false,
  "took": 15
}
```



Example to Use JMESPath Query Parameter:

The JMESPath query parameter `-q` is used to filter and retrieve specific data fields in the context of a query.

The following command search logs for `log-shipper` and applies the JMESPath query parameter `-q` to extract the message field from the hits.

```
bamctl logs query kubelog* "log-shipper" --size 5 -q hits.hits[]._source.mes
```

Response:

```
[
  "Beat name: eric-log-shipper-tpxtj",
  "Beat name: eric-log-shipper-l6s6c",
  "Beat name: eric-log-shipper-k2c59",
  "Beat name: eric-log-shipper-bpw75",
  "Beat name: eric-log-shipper-bpfmc"
]
```

Query to Search Logs collected and processed by BAM Opentelemetry

Note: BAM Opentelemetry processed logs are stored in Search Engine only when both `bamOpentelemetry` and `bamLegacyBackendsEnabled` macros are enabled to true.

This command provides an option to search logs collected and processed by BAM Opentelemetry. The logs follow the OpenTelemetry data model and have a different structure compared to legacy logs.

Also, the indices are created with a prefix `otlp-` by BAM Opentelemetry in Search Engine.

Example to get all indexes including BAM Opentelemetry processed logs:

```
bamctl logs get-all-indexes -q *.settings.index.provided_name
[
  "otlp-auditlog-2025.08.06",
  "otlp-kubelog-2025.08.06",
  "otlp-rawlogs-2025.08.06",
  ".bam-oam-dashboards_1",
  ".opendistro-job-scheduler-lock",
  "otlp-accesslog-2025.08.06",
  "otlp-adp-app-asi-logs-2025.08.06"
]
```



Example to query BAM Opentelemetry processed logs:



```

bamctl logs query otlp-kubelog* "Starting TLS listener at" --size 1
{
  "_shards": {
    "failed": 0,
    "skipped": 0,
    "successful": 5,
    "total": 5
  },
  "hits": {
    "hits": [
      {
        "_id": "cx8pfpgBM5AwG2CCFJC5",
        "_index": "otlp-kubelog-2025.08.06",
        "_score": 32.9193,
        "_source": {
          "@timestamp": "2025-08-06T06:54:32.296348116Z",
          "attributes": {
            "com.ericsson.adp.log_schema.version": "1.2.0",
            "com.ericsson.bam.log.file.path": "/var/log/pods/eferdsh_eric-bs",
            "com.ericsson.bam.logtag": "F",
            "com.ericsson.bam.restart_count": "0",
            "com.ericsson.bam.stream": "stdout",
            "com.ericsson.bam.structured": "true",
            "com.ericsson.common.time.iso_utc_offset": "2025-08-06T08:54:32.",
            "data_stream": {
              "dataset": "default",
              "namespace": "namespace",
              "type": "record"
            },
            "time": "2025-08-06T08:54:32.296348116+02:00"
          },
          "body": "[listener.go:47]: Starting TLS listener at: :5000, with c",
          "instrumentationScope": {},
          "observedTimestamp": "2025-08-06T06:54:38.264485981Z",
          "resource": {
            "k8s.container.name": "kmsgateway",
            "k8s.namespace.name": "eferdsh",
            "k8s.node.name": "node-100-77-174-121",
            "k8s.pod.name": "eric-bss-bam-kms-gateway-69bc7d6bf8-hvcpm",
            "k8s.pod.uid": "8e909a0d-3b44-452a-98d9-fc98d17520c4",
            "service.name": "eric-bss-bam-kms-gateway"
          },
          "severity": {
            "text": "info"
          }
        },
        "time": "2025-08-06T08:54:32.296348116+02:00"
      },
      {
        "_id": "cx8pfpgBM5AwG2CCFJC5",
        "_index": "otlp-kubelog-2025.08.06",
        "_score": 32.9193,
        "_source": {
          "@timestamp": "2025-08-06T06:54:32.296348116Z",
          "attributes": {
            "com.ericsson.adp.log_schema.version": "1.2.0",
            "com.ericsson.bam.log.file.path": "/var/log/pods/eferdsh_eric-bs",
            "com.ericsson.bam.logtag": "F",
            "com.ericsson.bam.restart_count": "0",
            "com.ericsson.bam.stream": "stdout",
            "com.ericsson.bam.structured": "true",
            "com.ericsson.common.time.iso_utc_offset": "2025-08-06T08:54:32.",
            "data_stream": {
              "dataset": "default",
              "namespace": "namespace",
              "type": "record"
            },
            "time": "2025-08-06T08:54:32.296348116+02:00"
          },
          "body": "[listener.go:47]: Starting TLS listener at: :5000, with c",
          "instrumentationScope": {},
          "observedTimestamp": "2025-08-06T06:54:38.264485981Z",
          "resource": {
            "k8s.container.name": "kmsgateway",
            "k8s.namespace.name": "eferdsh",
            "k8s.node.name": "node-100-77-174-121",
            "k8s.pod.name": "eric-bss-bam-kms-gateway-69bc7d6bf8-hvcpm",
            "k8s.pod.uid": "8e909a0d-3b44-452a-98d9-fc98d17520c4",
            "service.name": "eric-bss-bam-kms-gateway"
          },
          "severity": {
            "text": "info"
          }
        },
        "time": "2025-08-06T08:54:32.296348116+02:00"
      }
    ],
    "max_score": 32.9193,
    "total": {
      "relation": "eq",
      "value": 1234
    }
  },
  "timed_out": false,
  "took": 69
}

```



Search

This command provides option to search the logs based on the request body. The following example would get the results for time between yesterday and current time.

Usage:

```
bamctl logs search index [flags]
```

Flags:

```
-h, --help           help for search
-p, --prettyPrint    Reformat output
--size string
```

Method 1:

```
bamctl logs search kubelog* query.range.timestamp.gte: 'now-1d', .lt: now --size
```

Method 2:

```
bamctl logs search kubelog* < input.json
```

where input.json

```
{
  "query": {
    "range": {
      "timestamp": {
        "gte": "now-1d",
        "lt": "now"
      }
    }
  }
}
```

Response:



```
{
  "_shards": {
    "failed": 0,
    "skipped": 0,
    "successful": 10,
    "total": 10
  },
  "hits": {
    "hits": [
      {
        "_id": "kVnOGI4BhjM4wPL04VPM",
        "_index": "kubelog-2024.03.07",
        "_score": 1,
        "_source": {
          "@timestamp": "2024-03-07T12:06:33.305Z",
          "@version": "1",
          "extra_data": {
            "log_logger": "input.harvester",
            "log_origin": {
              "file_line": 310,
              "file_name": "log/harvester.go"
            }
          }
        },
        "logplane": "kubelog",
        "message": "Harvester started for paths: [/var/log/pods/namespace_...",
        "metadata": {
          "container_name": "logshipper",
          "namespace": "namespace",
          "node_name": "node-10-210-166-60",
          "pod_name": "eric-log-shipper-clmxs",
          "pod_uid": "071631f9-3f2e-4619-893e-3fb94fd798a0"
        },
        "service_id": "eric-log-shipper",
        "severity": "info",
        "timestamp": "2024-03-07T13:06:33.304+0100",
        "version": "1.2.0"
      }
    ],
    "max_score": 1,
    "total": {
      "relation": "gte",
      "value": 10000
    }
  },
  "timed_out": false,
  "took": 167
}
```



Example to Fetch Audit Logs From a Particular Service:

```
bamctl logs search auditlog* query.match.service_id: "eric-bss-bam-ref-tpg" --s
```

Response:



```
{
  "_shards": {
    "failed": 0,
    "skipped": 0,
    "successful": 5,
    "total": 5
  },
  "hits": {
    "hits": [
      {
        "_id": "I6yFnI0BTu8m9PoZmoCg",
        "_index": "auditlog-2024.02.12",
        "_score": 5.3682427,
        "_source": {
          "@timestamp": "2024-02-12T08:53:34.120Z",
          "@version": "1",
          "facility": "log audit",
          "logplane": "auditlog",
          "message": "[ User = Anonymous, IP = 192.168.225.238, Action = GET",
          "metadata": {
            "category": "BAM-tests",
            "container_name": "reftpg",
            "namespace": "namespace",
            "node_name": "node-10-210-166-48",
            "pod_name": "eric-bss-ref-tpg-7cd6bd7598-hmqx6",
            "pod_uid": "e4914e98-cd8e-4e65-a2b8-bc5ee06564f1"
          },
          "service_id": "eric-bss-bam-ref-tpg",
          "severity": "info",
          "timestamp": "2024-02-12T08:53:34.120Z",
          "version": "1.2.0"
        },
        "service_id": "eric-bss-bam-ref-tpg",
        "severity": "info",
        "timestamp": "2024-02-12T08:53:34.120Z",
        "version": "1.2.0"
      }
    ],
    "max_score": 5.3682427,
    "total": {
      "relation": "eq",
      "value": 498
    }
  },
  "timed_out": false,
  "took": 157
}
```

Example to Find Trace Logs with trace id- 12345 for Time Between Yesterday and Current Time from Container-xyz:



```
bamctl logs search tracelog* -p < input.json
```

where input.json

```
{
  "query": {
    {
      "bool": {
        "must": [
          {
            "range": {
              "timestamp": {
                "gte": "now-1d"
                "lt": "now"
              }
            }
          },
          {
            "match": {
              "metadata.container_name": "container-xyz"
            }
          },
          {
            "match_phrase_prefix": {
              "message": "SLM={\"TraceId\": \"12345\"}"
            }
          }
        ]
      }
    }
  }
}
```

Example to Use Exists Query to Search for Logs Containing a Specific Field:

Execute the following command to fetch kubelog with the specific field metadata.container_name.

```
bamctl logs search kubelog* query.exists.field: "metadata.container_name"
```

Response:



```

"_shards": {
  "failed": 0,
  "skipped": 0,
  "successful": 10,
  "total": 10
},
"hits": {
  "hits": [
    {
      "_id": "6IVcCI4Bi6GGegIvueFj",
      "_index": "kubelog-2024.03.04",
      "_score": 1,
      "_source": {
        "@timestamp": "2024-03-04T07:25:19.715Z",
        "@version": "1",
        "logplane": "kubelog",
        "message": "[adaptor.go:73]: Loading Rbac Policy (false) (299)",
        "metadata": {
          "container_name": "proxy",
          "namespace": "namespace",
          "node_name": "node-10-210-166-42",
          "pod_name": "eric-fh-alarm-handler-fi-rbac-proxy-66c7fcbddf-xj8m",
          "pod_uid": "5d8046f3-3b30-46e4-87ff-c001e30263b0"
        },
        "service_id": "eric-fh-alarm-handler-fi-rbac-proxy",
        "severity": "info",
        "timestamp": "2024-03-04T08:25:19.715+01:00",
        "version": "1.2.0"
      },
      "service_id": "eric-fh-alarm-handler-fi-rbac-proxy",
      "severity": "info",
      "timestamp": "2024-03-04T08:25:19.715+01:00",
      "version": "1.2.0"
    }
  ],
  "max_score": 1,
  "total": {
    "relation": "gte",
    "value": 10000
  }
},
"timed_out": false,
"took": 11
}

```

Example to Use Wildcard Queries to Search for Terms Matching a Specified Wildcard Pattern:

The following command is to search for terms in kubelog that starts with k and ends with g in the logplane field.



```
bamctl logs search kubelog* query.wildcard.logplane: k*g
```

Response:

```
{
  "_shards": {
    "failed": 0,
    "skipped": 0,
    "successful": 10,
    "total": 10
  },
  "hits": {
    "hits": [
      {
        "_id": "6IVcCI4Bi6GGegIvueFj",
        "_index": "kubelog-2024.03.04",
        "_score": 1,
        "_source": {
          "@timestamp": "2024-03-04T07:25:19.715Z",
          "@version": "1",
          "logplane": "kubelog",
          "message": "[adaptor.go:73]: Loading Rbac Policy (false) (299)",
          "metadata": {
            "container_name": "proxy",
            "namespace": "namespace",
            "node_name": "node-10-210-166-42",
            "pod_name": "eric-fh-alarm-handler-fi-rbac-proxy-66c7fcbddf-xj8mw",
            "pod_uid": "5d8046f3-3b30-46e4-87ff-c001e30263b0"
          },
          "service_id": "eric-fh-alarm-handler-fi-rbac-proxy",
          "severity": "info",
          "timestamp": "2024-03-04T08:25:19.715+01:00",
          "version": "1.2.0"
        }
      }
    ],
    "max_score": 1,
    "total": {
      "relation": "gte",
      "value": 10000
    }
  },
  "timed_out": false,
  "took": 9
}
```


**Example to Use Regexp Query to Search for Terms that Match a Regular Expression:**

The following command identifies entries starting with any uppercase or lowercase letter followed by `ric` in the `service_id` field.

```
bamctl logs search kubelog* query.regexp.service_id: "[A-Za-z]ric"
```

Response:



```
{
  "_shards": {
    "failed": 0,
    "skipped": 0,
    "successful": 10,
    "total": 10
  },
  "hits": {
    "hits": [
      {
        "_id": "EoVcCI4Bi6GGegIvueJj",
        "_index": "kubelog-2024.03.04",
        "_score": 1,
        "_source": {
          "@timestamp": "2024-03-04T07:26:19.522Z",
          "@version": "1",
          "logplane": "kubelog",
          "message": "[issuer_keys.go:79]: failed to get public key: 503 Service",
          "metadata": {
            "container_name": "proxy",
            "namespace": "namespace",
            "node_name": "node-10-210-166-42",
            "pod_name": "eric-fh-alarm-handler-fi-rbac-proxy-66c7fcbddf-xj8mw",
            "pod_uid": "5d8046f3-3b30-46e4-87ff-c001e30263b0"
          },
          "service_id": "eric-fh-alarm-handler-fi-rbac-proxy",
          "severity": "error",
          "timestamp": "2024-03-04T08:26:19.522+01:00",
          "version": "1.2.0"
        },
        "service_id": "eric-fh-alarm-handler-fi-rbac-proxy",
        "severity": "error",
        "timestamp": "2024-03-04T08:26:19.522+01:00",
        "version": "1.2.0"
      }
    ],
    "max_score": 1,
    "total": {
      "relation": "gte",
      "value": 10000
    }
  },
  "timed_out": false,
  "took": 15
}
```

Example to Prefix Query is to Search for Terms That Begin with Specified Prefix:

The following command is to search for terms in the **severity** field of kubelog that begin with the specified prefix err.



```
bamctl logs search kubelog* query.prefix.severity: "err"
```

Response:

```
{
  "_shards": {
    "failed": 0,
    "skipped": 0,
    "successful": 10,
    "total": 10
  },
  "hits": {
    "hits": [
      {
        "_id": "EoVcCI4Bi6GGegIvueJj",
        "_index": "kubelog-2024.03.04",
        "_score": 1,
        "_source": {
          "@timestamp": "2024-03-04T07:26:19.522Z",
          "@version": "1",
          "logplane": "kubelog",
          "message": "[issuer_keys.go:79]: failed to get public key: 503 Ser",
          "metadata": {
            "container_name": "proxy",
            "namespace": "namespace",
            "node_name": "node-10-210-166-42",
            "pod_name": "eric-fh-alarm-handler-fi-rbac-proxy-66c7fcbddf-xj8m",
            "pod_uid": "5d8046f3-3b30-46e4-87ff-c001e30263b0"
          },
          "service_id": "eric-fh-alarm-handler-fi-rbac-proxy",
          "severity": "error",
          "timestamp": "2024-03-04T08:26:19.522+01:00",
          "version": "1.2.0"
        }
      }
    ],
    "max_score": 1,
    "total": {
      "relation": "gte",
      "value": 10000
    }
  },
  "timed_out": false,
  "took": 15
}
```



Note: Use opensearch documentation to form complex query or search command based on filter requirements.

4.20 Metrics

Metrics provide options to query metrics information for counters through prometheus queries through RBAC Proxy of PM service.

Table 16 List of Commands

Command	Description
search	Query For PM metric information
search-range	Query For PM metric information over a range of time

Note: <prometheus-query> - For additional details of available queries, refer to the <https://prometheus.io/docs/prometheus/latest/querying/basics/>

Search

Usage:

```
bamctl metrics search <prometheus-query> (flags)
```

Flags:

```
-h, --help          help for search
--time string       Time in <rfc3339 | unix_timestamp> Evaluation timestamp
--timeout string    Evaluation timeout
```

Example

```
bamctl metrics search bamctl metrics search "count(cmm_http_request_duration_se
```

Response:



```
{
  "data": {
    "result": [
      {
        "metric": {},
        "value": [
          1594728798.732,
          "8"
        ]
      }
    ],
    "resultType": "vector"
  },
  "status": "success"
}
```

Search Range

Usage:

```
bamctl metrics search-range query start end step [flags]
```

Flags:

```
-h, --help            help for search-range
--timeout string      Evaluation timeout
```

```
bamctl metrics search-range "cmm_http_server_requests_total{app='eric-cm-med
```

Example Query by Metric Name(lt_jvm_mem_used_heap_percent):

```
bamctl metrics search lt_jvm_mem_used_heap_percent
```

Example Query by Label(kubernetes_name="eric-log-transformer" kubernetes_name="eric-log-transformer"):

```
bamctl metrics search {kubernetes_name=\"eric-log-transformer\"}
```

4.21 Topology Manager

Topology-manager provides multiple subcommands to handle locations.



```
bamctl topology-manager [command]
```

Table 17 List of Commands

Command	Description
delete-location	Delete location from topology manager.
remote-location	Distribute own location to remote Topology Manager.
list-locations	List all locations in topology manager

delete-location

This command provides option to delete a location from Topology Manager. It has one mandatory argument for the name of the location to be deleted.

Usage:

```
bamctl topology-manager delete-location <locationname> [flags]
```

Flags:

```
-h, --help    help for delete-location
```

Example

```
bamctl topology-manager delete-location Karlskrona01
```

Response:

```
{
  "message": "Successfully initiated detaching location Karlskrona01 from clu"
}
```

remote-location

This command provides option to distribute its own location to a specified remote Topology Manager. It has one mandatory argument for the remote Topology Manager FQDN.

Usage:



```
bamctl topology-manager remote-location fqdn:<remote-location>
```

Flags:

```
-h, --help    help for remote-location
```

Example

```
bamctl topology-manager remote-location fqdn:eric-bss-bam-topology-manager-s
```

Response:

```
{
  "message": "successfully added own location to remote Topology Manager"
}
```

list-locations

This command lists all locations registered in the topology manager.

Usage:

```
bamctl topology-manager list-locations
```

Flags:

```
-h, --help    help for list-locations
```

Response:



```
[
  {
    "address": {
      "fqdn": "eric-bss-bam-topology-manager-sync-rbac-proxy.cluster02.operator",
      "port": 443,
      "protocol": "https"
    },
    "clusterMembership": {
      "state": {
        "transitionedAt": "2024-10-01T22:15:01.309261505+02:00",
        "type": "Standalone"
      }
    },
    "clusterName": "c2",
    "dataCenter": "s2-dc",
    "instanceId": "40c5a14a-56af-49e3-814e-58399524b4c1",
    "name": "s2",
    "namespace": "bam",
    "region": "S-SE",
    "siteId": 2,
    "siteName": "Stockholm",
    "status": "Ready",
    "unreachableCounter": 0
  }
]
```

4.22 Extended Data Collection

xdc-bridge provides options to collect and download data from different targets. It has multiple subcommands.

```
bamctl xdc [command]
```

Table 18 List of Commands

Command	Description
collect-immediate	Executes collection and immediately returns the collected data as a file.
config-update	Update XDC config.
config-view	Retrieve valid xdc configuration.
delete-collection	Delete existing collection.
get-collection	Returns a status of existing collection.
get-collection-file	Returns a file of an existing collection.



Table 18 List of Commands

Command	Description
get-targets	Retrieve list of valid targets.
list-collections	Returns a list of existing collections.
new-data-collection	Triggers a data collection on a target in an asynchronous mode.
protected-file-view	Returns a protected file of an existing collection.

collect-immediate

This command provides the option of immediately collecting data from a target. It is recommended to use the command only if it is anticipated that a small amount of data would be collected.

Usage:

```
bamctl xdc collect-immediate <target>
```

Flags:

--extra string	Extra arguments for command json formatted
-h, --help	help for collect-immediate

```
bamctl xdc collect-immediate system
```

Response:

```
{
  "Downloaded filename": "xdc-collection-system.zip"
}
```

This command also supports an optional flag for providing additional arguments for the data collection process.

```
bamctl xdc collect-immediate <target> [flag]
```

Example

```
bamctl xdc collect-immediate eric-bss-cha-core-7cd886589d-fqkwc --extra "{ \"
```



The value of a flag is a string in JSON format wherein each field corresponds to a specific argument that is consumed by whichever collector subscribes to said argument.

delete-collection

This command provides option to delete a data collection.

Usage:

```
bamctl xdc delete-collection <collection-id>
```

Flags:

```
-h, --help    help for delete-collection
```

```
bamctl xdc delete-collection 61472513-a721-405c-b537-05ca08cfd6e4
```

Response:

```
"The collection id 61472513-a721-405c-b537-05ca08cfd6e4 is deleted successfully"
```

get-collection

This command provides option to get the status of a collection.

Usage:

```
bamctl xdc get-collection <collection-id>
```

Flags:

```
-h, --help    help for get-collection
```

Example

```
bamctl xdc get-collection 11fac209-75fa-4a53-969e-5a9bb48c4bb4
```

Response:



```
{
  "collectionType": "system",
  "end-time": "2024-09-13T13:30:45.76421035+02:00",
  "file": "/var/run/data/xdc-data/11fac209-75fa-4a53-969e-5a9bb48c4bb4.zip",
  "id": "11fac209-75fa-4a53-969e-5a9bb48c4bb4",
  "size": "13.97MB",
  "start-time": "2024-09-13T13:30:41.318559336+02:00",
  "status": "finished",
  "target": "system",
  "targetScope": "system"
}
```

get-collection-file

This command provides an option to download the data collected for a collection ID.

Usage:

```
bamctl xdc get-collection-file <collection-id>
```

Flags:

```
-h, --help    help for get-collection-file
```

```
bamctl xdc get-collection-file 2f43f35f-f1a8-4f83-bf93-3e3429709ca1
```

Response

```
{
  "Downloaded filename": "xdc-collection-2f43f35f-f1a8-4f83-bf93-3e3429709ca1.zip"
}
```

Get a decrypted file of the collection generated for ssl-key-logger type.

Usage:

```
bamctl xdc protected-file-view <collection-id>
```

Flags:

```
-h, --help    help for protected-file-view
```



```
bamctl xdc protected-file-view 2f43f35f-f1a8-4f83-bf93-3e3429709ca1
```

Response:

```
{  
  "Downloaded filename": "xdc-collection-2f43f35f-f1a8-4f83-bf93-3e3429709ca1.  
}
```

config-view

Retrieve valid XDC configuration.

```
bamctl xdc config-view
```

Aliases:

```
config-view, cv
```

Flags:

```
-h, --help    help for config-view
```

```
bamctl xdc config-view
```

Response:

```
{  
  "sslKeyLogger": {  
    "sslKeyLoggerDuration": 10,  
    "sslKeyLoggerMaxDuration": 60  
  }  
}
```

config-update

Update xdc configuration.

Usage:



```
bamctl xdc config-update <values json file or doted notation>
```

Flags:

```
-h, --help    help for config-update
```

```
bamctl xdc config-update .sslKeyLogger.sslKeyLoggerDuration: 11, .sslKeyLogg
```

Response:

```
"Configuration Updated."
```

get-targets

This command provides an option to list down the targets on which data collection can be triggered.

Usage:

```
bamctl xdc get-targets
```

Flags:

-h, --help	help for get-targets
--target-display string	Target display (short, name, long, full)
--target-scope string	Target scope (pod, workload, system)
--target-type string	Target type (plain, ssl-key-logger, spring)

Response:



```
{
  "targets": [
    {
      "collectionType": "system",
      "host": "CollectionCluster",
      "ip": "localhost",
      "method": "GET",
      "name": "system",
      "onDemandEndpoints": null,
      "partOf": "",
      "path": "/data-collection/v1/collections?target=system",
      "port": "3033",
      "scheme": "https",
      "targetScope": "system"
    }
  ]
}
```

list-collections

This command provides an option to list down the collections and its status triggered using the asynchronous mode.

Usage:

```
bamctl xdc list-collections
```

Flags:

```
-h, --help    help for list-collections
```

Response:



```
{
  "collections": [
    {
      "collectionType": "system",
      "end-time": "2023-02-01T15:52:31.844899096Z",
      "file": "/var/run/data/xdc-data/2f43f35f-f1a8-4f83-bf93-3e3429709ca1.z",
      "id": "2f43f35f-f1a8-4f83-bf93-3e3429709ca1",
      "size": "20.03MB",
      "start-time": "2023-02-01T15:52:16.749012799Z",
      "status": "finished",
      "target": "system",
      "targetScope": "system"
    }
  ]
}
```

new-data-collection

This command provides an option to trigger a data collection on a target in an asynchronous mode. It is recommended to use this command if it is anticipated to collect a large amount of data from a target.

Usage:

```
bamctl xdc new-data-collection <target>
```

Flags:

--extra string	Extra arguments for command json formatted
-h, --help	help for new-data-collection

```
bamctl xdc new-data-collection system
```

Response:

```
{
  "id": "6f33cc17-b7bb-4fb3-991b-0d52d3bbc2e6"
}
```

This command also supports an optional flag for providing additional arguments to the data collection process.

```
bamctl xdc new-data-collection <target> [flag]
```



Example

```
bamctl xdc new-data-collection eric-bss-cha-core-7cd886589d-fqkwc --extra "{\
```

The value of a flag is a string in JSON format wherein each field corresponds to a specific argument that would be consumed by whichever collector subscribes to said argument.

Note: XDC collection commands support including a timestamp in the downloaded file names. For details, refer to [Timestamped Output File Naming](#).

4.22.1 Timestamped Output File Naming

When the `config.collection.outputFileNameFormat` is set to `timestamped` in the XDC Bridge configuration, the collection output filenames include a UTC timestamp in the format `YYYYMMDDTHHMMSSZ`.

This helps the user identify when a collection was created and distinguish files from repeated collections.

For more information about this configuration, refer to the [Extended Data Collection \(XDC\) Configuration Parameters](#).

The following examples show the timestamped filename format for different collection types.

Collect immediate:system

```
bamctl xdc collect-immediate system
{"Downloaded filename":"xdc-collection_20260422T141742Z_system.zip"}
```

Collect immediate:pod

```
bamctl xdc collect-immediate eric-bss-bam-topology-manager-69fd479bd6-r6s78
{"Downloaded filename":"xdc-collection_20260422T141807Z_eric-bss-bam-topology-m
```

Collect immediate:workload

```
bamctl xdc collect-immediate eric-bss-bam-domain-proxy_plain
{"Downloaded filename":"xdc-collection_20260422T141846Z_eric-bss-bam-domain-pro
```




Asynchronous collections downloaded with `get-collection-file` also include a short collection ID (first eight characters of the collection UUID) at the end of the filename.

Collect `async:system`

```
bamctl xdc new-data-collection system
```

Response:

```
{
  "id": "69b0b0a3-5dd5-4522-99f9-32c935e34292"
}
```

```
bamctl xdc get-collection-file 69b0b0a3-5dd5-4522-99f9-32c935e34292
{"Downloaded filename":"xdc-collection_20260422T141931Z_system_69b0b0a3.zip"}
```

Collect `async:pod`

```
bamctl xdc new-data-collection eric-bss-bam-topology-manager-69fd479bd6-r6s7
```

Response:

```
{
  "id": "851ad9bd-97d6-4a86-b7ad-f118b6bdb521"
}
```

```
bamctl xdc get-collection-file 851ad9bd-97d6-4a86-b7ad-f118b6bdb521
{"Downloaded filename":"xdc-collection_20260422T142016Z_eric-bss-bam-topolog
```

Collect `async:workload`

```
bamctl xdc new-data-collection eric-bss-bam-domain-proxy_plain
```

Response:



```
{
  "id": "d5841376-d57d-4907-b44c-649725e6d44f"
}
```

```
bamctl xdc get-collection-file d5841376-d57d-4907-b44c-649725e6d44f
{"Downloaded filename":"xdc-collection_20260422T142103Z_eric-bss-bam-domain-pro
```

4.23 Trace Management

Commands to manage Trace Jobs for services that are enlisted in BAM Operations Provider's Trace Management.

Services need to be enlisted with service-specific interfaces and supported criteria names.

Table 19 Available Commands

Command	Description
create-trace-job	Create a Trace Job.
delete-trace-job	Delete a trace job.
delete-trace-jobs	Delete all trace jobs matching an interface (and optionally a criteria type)
get-trace-job	Retrieve a trace job.
get-trace-logs	Retrieve trace logs.
get-trace-config	Retrieve trace management configuration.
list-trace-jobs	List all trace jobs (optionally matching an interface and criteria type)
start-trace-job	Immediately start a Trace Job (unless already started)
stop-trace-job	Immediately stop a Trace Job (unless already stopped)
update-trace-job	Update a Trace Job.
update-trace-config	Update existing trace management configuration.

create-trace-job

This command creates a trace job configuration.

Usage:



```
bamctl trace-management create-trace-job<input.json
```

Flags:

```
-h, --help    help for create-trace-job
```

where input.json

```
{
  "startTime": "2023-09-05T16:15:59+02:00",
  "stopTime": "2023-10-05T16:15:27+02:00",
  "criteriaType": "ContractId",
  "interface": "CHA-PER-ACT-INT",
  "traceLevel": 0,
  "traceJobDescription": "trace job for contract id",
  "coverageType": "LOCAL",
  "criteriaValue": "490A1CFB762F410BB10914181FA94135"
}
```

Response:

```
{
  "startTime": "2023-09-05T16:15:59+02:00",
  "stopTime": "2023-10-05T16:15:27+02:00",
  "criteriaType": "ContractId",
  "interface": "CHA-PER-ACT-INT",
  "traceLevel": 0,
  "traceJobDescription": "trace job for contract id",
  "coverageType": "LOCAL",
  "criteriaValue": "490A1CFB762F410BB10914181FA94135",
  "trace-id": "c54fb2d4-2078-4a36-923d-a6295cf18da7"
}
```

delete-trace-job

This command deletes the trace job for the given trace-id.

Usage

```
bamctl trace-management delete-trace-job trace-id
```

Flags:

```
-h, --help    help for delete-trace-job
```



```
bamctl trace-management delete-trace-job c54fb2d4-2078-4a36-923d-a6295cf18da7
```

delete-trace-jobs

This command deletes all trace jobs matching an interface (and optionally a criteria type).

```
bamctl trace-management delete-trace-jobs interface [--criteria-type]
```

Flags:

<code>--criteria-type string</code>	Specify the trace criteria type to filter when c
<code>-h, --help</code>	help for delete-trace-jobs

```
bamctl trace-management delete-trace-jobs CHA-PER-ACT-INT
```

```
bamctl trace-management delete-trace-jobs CHA-PER-ACT-INT --criteria-type Contr
```

get-trace-job

This command retrieves a trace job for the given trace-id.

Usage:

```
bamctl trace-management get-trace-job trace-id
```

Flags:

<code>-h, --help</code>	help for get-trace-job
-------------------------	------------------------

```
bamctl trace-management get-trace-job c54fb2d4-2078-4a36-923d-a6295cf18da7
```

Response:



```
{
  "traceJobs": [
    {
      "coverageType": "LOCAL",
      "criteriaType": "ContractId",
      "criteriaValue": "490A1CFB762F410BB10914181FA94135",
      "interface": "CHA-PER-ACT-INT",
      "startTime": "2023-09-05T16:15:59.1111+02:00",
      "stopTime": "2023-10-05T16:15:27+02:00",
      "traceId": "2f455d42-974b-4c0f-96b6-d8aff99cb32b",
      "traceJobDescription": "this is valid trace job",
      "traceLevel": 0
    }
  ]
}
```

get-trace-logs

This command retrieves trace logs for a given trace-id.

Usage:

```
bamctl trace-management get-trace-logs trace-id [-p]
```

Flags:

-h, --help	help for get-trace-logs
-p, --prettyPrint	Reformat output

```
bamctl trace-management get-trace-logs 8f9890d2-de52-444b-bc40-21653313dd26
```

Response:



```
{
  "hits": {
    "hits": [
      {
        "_source": {
          "@timestamp": "2023-05-10T07:56:47.480Z",
          "@version": "1",
          "extra_data": {
            "trace": {
              "traceId": "8f9890d2-de52-444b-bc40-21653313dd26"
            }
          },
          "facility": "log trace",
          "kubernetes": {
            "replicaset": {
              "name": "eric-bss-ref-tpg-bbb57c6f4"
            }
          },
          "logplane": "tracelog",
          "message": "8f9890d2-de52-444b-bc40-21653313dd26: Example of extra_
This log contains: new lines, tabs and double quotes in the message"
          "metadata": {
            "category": "TRACE",
            "container_name": "reftpg",
            "namespace": "bam-2-system",
            "node_name": "controlplane-bam-master-3",
            "pod_name": "eric-bss-ref-tpg-bbb57c6f4-7j68n",
            "pod_uid": "31d5f4a0-2749-4a3b-955b-0d2c0ec40d9c"
          },
          "service_id": "eric-bss-bam-ref-tpg",
          "severity": "info",
          "timestamp": "2023-05-10T07:56:47.480Z",
          "version": "1.2.0"
        }
      }
    ]
  }
}
```

Optionally with **-p (--prettyPrint)** flag, the **message** in the log output would be formatted.

```
bamctl trace-management get-trace-logs 8f9890d2-de52-444b-bc40-21653313dd26
```



```
{
  "hits": {
    "hits": [
      {
        "_source": {
          "@timestamp": "2023-05-10T07:56:47.480Z",
          "@version": "1",
          "extra_data": {
            "trace": {
              "traceId": "8f9890d2-de52-444b-bc40-21653313dd26"
            }
          },
          "facility": "log trace",
          "kubernetes": {
            "replicaset": {
              "name": "eric-bss-ref-tpg-bbb57c6f4"
            }
          },
          "logplane": "tracelog",
          "message": "8f9890d2-de52-444b-bc40-21653313dd26: Example of ext
This log contains: new lines, tabs, and double quotes in the mes
These must be replaced as per the trace log appender replacers d
          "metadata": {
            "category": "TRACE",
            "container_name": "reftpg",
            "namespace": "bam-2-system",
            "node_name": "controlplane-bam-master-3",
            "pod_name": "eric-bss-ref-tpg-bbb57c6f4-7j68n",
            "pod_uid": "31d5f4a0-2749-4a3b-955b-0d2c0ec40d9c"
          },
          "service_id": "eric-bss-bam-ref-tpg",
          "severity": "info",
          "timestamp": "2023-05-10T07:56:47.480Z",
          "version": "1.2.0"
        }
      }
    ]
  }
}
```

get-trace-config

This command retrieves trace management configuration in use.

Usage:



```
bamctl trace-management get-trace-config
```

Flags:

```
-h, --help    help for get-trace-config
```

Response:

```
{
  "activeTraceJobsThreshold": 10,
  "inactiveJobsRetentionPeriod": 2880,
  "traceLogsIndex": "tracelog",
  "traceLogsSize": 100
}
```

where

- `activeTraceJobsThreshold` - Number of active trace jobs per interface
- `inactiveJobsRetentionPeriod` - Time elapse after which expired trace jobs are eligible for cleanup
- `traceLogsIndex` - Name of search engine log index used to fetch trace logs
- `traceLogsSize` - Trace log size in a search engine query

list-trace-jobs

This command retrieves all the trace jobs available in the system (optionally matching an interface and criteria type).

Usage:

```
bamctl trace-management list-trace-jobs [flags]
```

Flags:

```
--criteria-type string  Specify the trace criteria type to filter when l
-h, --help              help for list-trace-jobs
--interface string      Specify the trace interface to filter when listi
```

Example 1: List all jobs

```
bamctl trace-management list-trace-jobs
```

Response:



```
{
  "traceJobs": [
    {
      "coverageType": "LOCAL",
      "criteriaType": "ContractId",
      "criteriaValue": "490A1CFB762F410BB10914181FA94135",
      "interface": "CHA-PER-ACT-INT",
      "startTime": "2023-09-05T16:15:59.1111+02:00",
      "stopTime": "2023-10-05T16:15:27+02:00",
      "traceId": "eb2dfb4-9abc-4c15-b0dd-b120b8b94729",
      "traceJobDescription": "ContractId trace job",
      "traceLevel": 0
    },
    {
      "coverageType": "LOCAL",
      "criteriaType": "MSISDN",
      "criteriaValue": "552337005",
      "interface": "CHA-PER-ACT-INT",
      "startTime": "2023-09-05T16:15:59.1111+02:00",
      "stopTime": "2023-10-05T16:15:27+02:00",
      "traceId": "37026b31-a15e-4518-8fa5-74eef808c003",
      "traceJobDescription": "CustomerId trace job",
      "traceLevel": 0
    }
  ]
}
```

Example 2: List jobs with criteria

```
bamctl trace-management list-trace-jobs --criteria-type ContractId
```

Response:



```
{
  "traceJobs": [
    {
      "coverageType": "LOCAL",
      "criteriaType": "ContractId",
      "criteriaValue": "490A1CFB762F410BB10914181FA94135",
      "interface": "CHA-PER-ACT-INT",
      "startTime": "2023-09-05T16:15:59.1111+02:00",
      "stopTime": "2023-10-05T16:15:27+02:00",
      "traceId": "eb2dffb4-9abc-4c15-b0dd-b120b8b94729",
      "traceJobDescription": "ContractId trace job",
      "traceLevel": 0
    }
  ]
}
```

Example 3: List jobs with criteria and interface

```
bamctl trace-management list-trace-jobs --criteria-type ContractId --interface
```

Response:

```
{
  "traceJobs": [
    {
      "coverageType": "LOCAL",
      "criteriaType": "ContractId",
      "criteriaValue": "490A1CFB762F410BB10914181FA94135",
      "interface": "CHA-PER-ACT-INT",
      "startTime": "2023-09-05T16:15:59.1111+02:00",
      "stopTime": "2023-10-05T16:15:27+02:00",
      "traceId": "eb2dffb4-9abc-4c15-b0dd-b120b8b94729",
      "traceJobDescription": "ContractId trace job",
      "traceLevel": 0
    }
  ]
}
```

start-trace-job

This command immediately starts a trace job for a given trace-id, if the trace job is not already started. The startTime for the given trace job would be updated to the current time.

Usage:



```
bamctl trace-management start-trace-job trace-id
```

Flags:

```
-h, --help    help for start-trace-job
```

```
bamctl trace-management start-trace-job eb2dfb4-9abc-4c15-b0dd-b120b8b94729
```

Response:

```
{
  "coverageType": "LOCAL",
  "criteriaType": "ContractId",
  "criteriaValue": "490A1CFB762F410BB10914181FA94135",
  "interface": "CHA-PER-ACT-INT",
  "startTime": "2023-05-24T10:43:52+02:00",
  "stopTime": "2023-10-05T16:15:27+02:00",
  "traceId": "eb2dfb4-9abc-4c15-b0dd-b120b8b94729",
  "traceJobDescription": "this is valid trace job",
  "traceLevel": 0
}
```

stop-trace-job

This command immediately stops the trace job for a given `trace-id`, if the trace job is not already stopped. The `stopTime` for the given trace job would be updated to the current time.

Usage:

```
bamctl trace-management stop-trace-job trace-id
```

Flags:

```
-h, --help    help for stop-trace-job
```

```
bamctl trace-management stop-trace-job eb2dfb4-9abc-4c15-b0dd-b120b8b94729
```



```
{
  "coverageType": "LOCAL",
  "criteriaType": "ContractId",
  "criteriaValue": "490A1CFB762F410BB10914181FA94135",
  "interface": "CHA-PER-ACT-INT",
  "startTime": "2023-05-24T10:43:52+02:00",
  "stopTime": "2023-10-05T16:15:27+02:00",
  "traceId": "eb2dff4-9abc-4c15-b0dd-b120b8b94729",
  "traceJobDescription": "this is valid trace job",
  "traceLevel": 0
}
```

update-trace-job

This command updates the `startTime`, `stopTime` or both for the given job as per the input provided for future or already running jobs.

Usage:

```
bamctl trace-management update-trace-job trace-id .startTime:<startTime>,.stopTime:<stopTime>
```

Flags:

<code>-h, --help</code>	help for update-trace-job
<code>--x-action-indicator string</code>	The action indicator to perform

Example 1: With both Start time and Stop time

```
bamctl trace-management update-trace-job eb2dff4-9abc-4c15-b0dd-b120b8b94729 .startTime:2023-06-05T16:15:27+02:00,.stopTime:2023-07-10T16:15:27+02:00
```

Response:

```
{
  "coverageType": "LOCAL",
  "criteriaType": "ContractId",
  "criteriaValue": "490A1CFB762F410BB10914181FA94135",
  "interface": "CHA-PER-ACT-INT",
  "startTime": "2023-06-05T16:15:27+02:00",
  "stopTime": "2023-07-10T16:15:27+02:00",
  "traceId": "eb2dff4-9abc-4c15-b0dd-b120b8b94729",
  "traceJobDescription": "this is valid trace job",
  "traceLevel": 0
}
```

Example 2: With Start time



```
bamctl trace-management update-trace-job eb2dff4-9abc-4c15-b0dd-b120b8b9472
```

Response:

```
{
  "coverageType": "LOCAL",
  "criteriaType": "ContractId",
  "criteriaValue": "490A1CFB762F410BB10914181FA94135",
  "interface": "CHA-PER-ACT-INT",
  "startTime": "2023-06-06T16:15:27+02:00",
  "stopTime": "2023-07-10T16:15:27+02:00",
  "traceId": "eb2dff4-9abc-4c15-b0dd-b120b8b94729",
  "traceJobDescription": "this is valid trace job",
  "traceLevel": 0
}
```

Example 3: With Stop time

```
bamctl trace-management update-trace-job eb2dff4-9abc-4c15-b0dd-b120b8b9472
```

Response:

```
{
  "coverageType": "LOCAL",
  "criteriaType": "ContractId",
  "criteriaValue": "490A1CFB762F410BB10914181FA94135",
  "interface": "CHA-PER-ACT-INT",
  "startTime": "2023-06-05T16:15:27+02:00",
  "stopTime": "2023-07-11T16:15:27+02:00",
  "traceId": "eb2dff4-9abc-4c15-b0dd-b120b8b94729",
  "traceJobDescription": "this is valid trace job",
  "traceLevel": 0
}
```

update-trace-config

This command updates existing trace management configuration.

Usage:



```
bamctl trace-management update-trace-config .activeTraceJobsThreshold:<activeTr
```

Flags:

```
-h, --help    help for update-trace-config
```

```
bamctl trace-management update-trace-config .activeTraceJobsThreshold:11
```

Response:

```
{
  "activeTraceJobsThreshold": 11,
  "inactiveJobsRetentionPeriod": 2880,
  "traceLogsIndex": "tracelog",
  "traceLogsSize": 100
}
```